DocGuard AI — Notebook 2: Metadata MLP + Font CRNN

Part A — Metadata MLP: Detects forgery from PDF/image metadata

- Date mismatches (doc says 2022, created 2025)
- Software anachronisms (Photoshop on a 'scanned' doc)
- Wiped author fields, XMP vs DocInfo disagreement

Part B — Font CRNN: CNN + BiLSTM detects font inconsistencies

- Two fonts in same line (original + inserted text)
- Font version postdates document claimed date
- Kerning/spacing statistical outliers

```
# ── Install ──────────────────────────────────────────────────────────
!pip install -q torch torchvision scikit-learn PyMuPDF pillow numpy pandas \
               matplotlib seaborn tqdm

from google.colab import drive
drive.mount('/content/drive')
SAVE_DIR = '/content/drive/MyDrive/docguard_models'

import torch, torch.nn as nn
import numpy as np, pandas as pd
from pathlib import Path
print('Ready.')
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 24.9/24.9 MB 40.0 MB/s eta 0:00:00
Mounted at /content/drive
Ready.
```

```
# ═══════════════════════════════════════════════════════════════════
# PART A — METADATA MLP
# ═══════════════════════════════════════════════════════════════════

# ── Cell A1: Generate synthetic metadata features dataset ──────────
import random
import json
import numpy as np
from datetime import datetime, timedelta
```

```
random.seed(42)
np.random.seed(42)

PHOTOSHOP_VERSIONS = {
    2015: 'Adobe Photoshop CC 2015', 2016: 'Adobe Photoshop CC 2016',
    2017: 'Adobe Photoshop CC 2017', 2018: 'Adobe Photoshop CC 2018',
    2019: 'Adobe Photoshop CC 2019', 2020: 'Adobe Photoshop 2020',
    2021: 'Adobe Photoshop 2021',    2022: 'Adobe Photoshop 2022',
    2023: 'Adobe Photoshop 2023',    2024: 'Adobe Photoshop 2024',
}

SCANNER_TOOLS = [
    'HP ScanJet Pro', 'Epson Scan 2', 'Canon IJ Scan Utility',
    'NAPS2', 'VueScan', 'Adobe Acrobat Scanner', 'Fujitsu ScanSnap'
]

def make_authentic_metadata(doc_year_range=(2018, 2024)):
    """Generate realistic metadata for an AUTHENTIC document."""
    doc_year  = random.randint(*doc_year_range)
    doc_month = random.randint(1, 12)

    # Authentic: scanned shortly after doc date
    scan_delay_days = random.randint(0, 180)
    scan_date = datetime(doc_year, doc_month, 1) + timedelta(days=scan_delay_days)

    return {
        'creation_year':            scan_date.year,
        'creation_month':           scan_date.month,
        'modified_year':            scan_date.year,
        'modified_month':           scan_date.month,
        'creation_modified_delta':  random.randint(0, 5),
        'content_date_year':        doc_year,
        'content_creation_delta':   max(0, (scan_date.year - doc_year) * 12 + (scan_date.month - doc_month)),
        'producer_is_photoshop':    0,
        'producer_is_scanner':      1,
        'producer_version_year':    scan_date.year,
        'software_year_vs_doc_year': 0,  # software existed when doc was made
        'author_blank':             random.choice([0, 0, 0, 1]),
        'author_length':            random.randint(5, 30),
        'xmp_docinfo_mismatch':     0,
        'has_javascript':           0,
        'font_version_postdates_doc': 0,
        'mixed_font_types':         0,
        'compression_layers':       1,
        'label': 0  # AUTHENTIC
    }
```

```
def make_forged_metadata(forgery_type='date_mismatch'):
    """Generate metadata for a FORGED document with specific forgery."""

    if forgery_type == 'date_mismatch':  # YOUR EXACT EXAMPLE
        # Doc content says 2022, but PDF created 2025
        content_year  = random.randint(2015, 2022)
        creation_year = random.randint(2023, 2025)
        return {
            'creation_year':            creation_year,
            'creation_month':           random.randint(1, 12),
            'modified_year':            creation_year,
            'modified_month':           random.randint(1, 12),
            'creation_modified_delta':  random.randint(0, 2),
            'content_date_year':        content_year,
            'content_creation_delta':   (creation_year - content_year) * 12,  # LARGE = red flag
            'producer_is_photoshop':    random.choice([0, 1]),
            'producer_is_scanner':      0,
            'producer_version_year':    creation_year,
            'software_year_vs_doc_year': creation_year - content_year,  # KEY SIGNAL
            'author_blank':             random.choice([1, 1, 0]),  # often wiped
            'author_length':            random.randint(0, 10),
            'xmp_docinfo_mismatch':     random.choice([0, 1]),
            'has_javascript':           0,
            'font_version_postdates_doc': 1,  # Calibri 2023 on 2019 doc
            'mixed_font_types':         random.choice([0, 1]),
            'compression_layers':       random.choice([1, 2]),
            'label': 1  # FORGED
        }

    elif forgery_type == 'photoshop_tell':
        year = random.randint(2018, 2023)
        return {
            'creation_year': year, 'creation_month': random.randint(1,12),
            'modified_year': year, 'modified_month': random.randint(1,12),
            'creation_modified_delta': random.randint(0,30),
            'content_date_year': year,
            'content_creation_delta': random.randint(0, 30),
            'producer_is_photoshop': 1,     # BIG RED FLAG on a document
            'producer_is_scanner': 0,
            'producer_version_year': year,
            'software_year_vs_doc_year': 0,
            'author_blank': 1,
            'author_length': 0,
            'xmp_docinfo_mismatch': random.choice([0, 1]),
            'has_javascript': 0,
            'font_version_postdates_doc': 0,
            'mixed_font_types': 1,
```

```
            'compression_layers': 1,
            'label': 1
        }

    elif forgery_type == 'wiped_metadata':
        year = random.randint(2018, 2024)
        return {
            'creation_year': year, 'creation_month': random.randint(1,12),
            'modified_year': year + random.randint(0,2),
            'modified_month': random.randint(1,12),
            'creation_modified_delta': random.randint(100, 1000),
            'content_date_year': year - random.randint(2, 5),
            'content_creation_delta': random.randint(24, 60),
            'producer_is_photoshop': 0,
            'producer_is_scanner': 0,  # tool wiped
            'producer_version_year': 0,
            'software_year_vs_doc_year': random.randint(2, 7),
            'author_blank': 1,          # author wiped
            'author_length': 0,
            'xmp_docinfo_mismatch': 1,  # two layers disagree
            'has_javascript': 0,
            'font_version_postdates_doc': random.choice([0, 1]),
            'mixed_font_types': random.choice([0, 1]),
            'compression_layers': 2,
            'label': 1
        }

# Generate 20,000 metadata records (10k authentic, 10k forged)
records = []
for _ in range(10000):
    records.append(make_authentic_metadata())

forgery_types = ['date_mismatch', 'photoshop_tell', 'wiped_metadata']
for i in range(10000):
    ft = forgery_types[i % len(forgery_types)]
    records.append(make_forged_metadata(ft))

random.shuffle(records)
df = pd.DataFrame(records)
df.to_csv('/content/metadata_dataset.csv', index=False)
print(f'Metadata dataset: {len(df)} records')
print(f'Label distribution:\n{df["label"].value_counts()}')
df.head()
```

```
Metadata dataset: 20000 records
Label distribution:
label
1    10000
0    10000
Name: count, dtype: int64
```

	creation_year	creation_month	modified_year	modified_month	creation_modified_delta	content_date_year	content_creation_delta	producer_is_photoshop
0	2024	5	2024	8	156	2022	27	0
1	2020	11	2021	1	618	2017	37	0
2	2023	5	2023	9	2	2022	12	0
3	2023	10	2023	10	1	2023	2	0
4	2021	3	2021	3	1	2021	0	0

```
# ── Cell A2: Metadata MLP Architecture + Training ────────────────
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, roc_auc_score

FEATURE_COLS = [c for c in df.columns if c != 'label']

X = df[FEATURE_COLS].values.astype(np.float32)
y = df['label'].values.astype(np.int64)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

scaler  = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

device  = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

train_ds = TensorDataset(
    torch.tensor(X_train), torch.tensor(y_train))
test_ds  = TensorDataset(
    torch.tensor(X_test),  torch.tensor(y_test))

train_loader = DataLoader(train_ds, batch_size=256, shuffle=True)
```

```
test_loader  = DataLoader(test_ds,  batch_size=256)


class MetadataMLP(nn.Module):
    """
    5-layer MLP for metadata-based forgery detection.
    Input: 18 metadata features
    Output: 2 classes (authentic / forged)

    Key features it learns:
    - Large content_creation_delta = backdated doc
    - producer_is_photoshop=1 on scan = red flag
    - software_year_vs_doc_year >> 0 = anachronism
    """
    def __init__(self, input_dim=18, num_classes=2):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 128), nn.BatchNorm1d(128), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(128, 256),       nn.BatchNorm1d(256), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(256, 128),       nn.BatchNorm1d(128), nn.ReLU(), nn.Dropout(0.2),
            nn.Linear(128, 64),        nn.BatchNorm1d(64),  nn.ReLU(),
            nn.Linear(64,  num_classes)
        )

    def forward(self, x): return self.net(x)


meta_model = MetadataMLP(input_dim=len(FEATURE_COLS)).to(device)
optimizer  = torch.optim.Adam(meta_model.parameters(), lr=1e-3)
criterion  = nn.CrossEntropyLoss()

print('Training Metadata MLP...')
for epoch in range(1, 51):
    meta_model.train()
    for xb, yb in train_loader:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        loss = criterion(meta_model(xb), yb)
        loss.backward()
        optimizer.step()

    if epoch % 10 == 0:
        meta_model.eval()
        correct = total = 0
        with torch.no_grad():
            for xb, yb in test_loader:
                preds   = meta_model(xb.to(device)).argmax(1).cpu()
```

```
                correct += (preds == yb).sum().item()
                total   += len(yb)
        print(f'  Epoch {epoch}: test acc = {correct/total*100:.1f}%')

# Final evaluation
meta_model.eval()
all_preds, all_probs, all_labels = [], [], []
with torch.no_grad():
    for xb, yb in test_loader:
        out   = meta_model(xb.to(device))
        probs = torch.softmax(out, 1)[:, 1].cpu().numpy()
        preds = out.argmax(1).cpu().numpy()
        all_preds.extend(preds); all_probs.extend(probs); all_labels.extend(yb.numpy())

print('\n=== Metadata MLP Test Results ===')
print(classification_report(all_labels, all_preds, target_names=['Authentic', 'Forged']))
print(f'AUC-ROC: {roc_auc_score(all_labels, all_probs):.4f}')

import joblib
torch.save(meta_model.state_dict(), f'{SAVE_DIR}/metadata_mlp.pt')
joblib.dump(scaler,                 f'{SAVE_DIR}/metadata_scaler.pkl')
json.dump(FEATURE_COLS, open(f'{SAVE_DIR}/metadata_features.json', 'w'))
print('Metadata MLP saved!')
```

```
Training Metadata MLP...
  Epoch 10: test acc = 100.0%
  Epoch 20: test acc = 100.0%
  Epoch 30: test acc = 100.0%
  Epoch 40: test acc = 100.0%
  Epoch 50: test acc = 100.0%

=== Metadata MLP Test Results ===
              precision    recall  f1-score   support

   Authentic       1.00      1.00      1.00      2000
      Forged       1.00      1.00      1.00      2000

    accuracy                           1.00      4000
   macro avg       1.00      1.00      1.00      4000
weighted avg       1.00      1.00      1.00      4000

AUC-ROC: 1.0000
Metadata MLP saved!
```

```
# ═══════════════════════════════════════════════════════════════
# PART B — FONT CRNN (CNN + BiLSTM)
# Detects font inconsistencies in document text lines
# ═══════════════════════════════════════════════════════════════
```

```
class FontCRNN(nn.Module):
    """
    Convolutional Recurrent Neural Network for font consistency analysis.

    Architecture:
    - CNN backbone: extracts visual features from text line images
    - Feature map reshaped into time sequence
    - BiLSTM reads left-to-right (and right-to-left) across the line
    - Classifier determines: consistent font vs mixed/replaced font

    Why CRNN for fonts?
    - Text lines have a natural LEFT→RIGHT sequence
    - Font inconsistency is LOCAL (one word differs from rest)
    - BiLSTM can see context: 'this character looks different from neighbours'
    - CNN sees the visual glyph; LSTM sees the sequence context
    """
    def __init__(self, num_classes=2):
        super().__init__()

        # CNN: processes text line image → feature maps
        self.cnn = nn.Sequential(
            # Block 1: 1→32 channels
            nn.Conv2d(1, 32, kernel_size=3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2, 2),   # H/2 x W/2
            # Block 2: 32→64
            nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2, 2),   # H/4 x W/4
            # Block 3: 64→128
            nn.Conv2d(64, 128, kernel_size=3, padding=1), nn.ReLU(),
            nn.AdaptiveAvgPool2d((1, None)),  # Collapse height to 1
        )

        # BiLSTM: reads the W-dimensional sequence left and right
        self.lstm = nn.LSTM(
            input_size  = 128,
            hidden_size = 256,
            num_layers  = 2,
            bidirectional = True,
            batch_first = True,
            dropout     = 0.3
        )

        # Classifier on final hidden state
        self.classifier = nn.Sequential(
            nn.Linear(256 * 2, 128),  # *2 for bidirectional
            nn.ReLU(),
```

```
            nn.Dropout(0.4),
            nn.Linear(128, num_classes)
        )

    def forward(self, x):
        # x: (B, 1, H, W) — grayscale text line image
        features = self.cnn(x)           # (B, 128, 1, W')
        features = features.squeeze(2)   # (B, 128, W')
        features = features.permute(0, 2, 1)  # (B, W', 128) = sequence
        lstm_out, _ = self.lstm(features)     # (B, W', 512)
        # Use final timestep (captures full sequence context)
        final = lstm_out[:, -1, :]            # (B, 512)
        return self.classifier(final)


crnn = FontCRNN(num_classes=2).to(device)
total = sum(p.numel() for p in crnn.parameters() if p.requires_grad)
print(f'Font CRNN parameters: {total:,}')

# Training uses synthetic text line images generated by the forgery generator
# For Colab: text line images are 32px high x 256px wide, grayscale
# authentic lines: uniform font throughout
# forged lines:    different font in 1-3 word region
print('Font CRNN architecture ready.')
print('Train this on your generated dataset text line crops.')
print('See notebook for full training loop (same pattern as ELA-CNN).')

# Save architecture
torch.save(crnn.state_dict(), f'{SAVE_DIR}/font_crnn_init.pt')
print('CRNN saved (untrained init weights — run full training loop to train).')
```

```
Font CRNN parameters: 2,526,082
Font CRNN architecture ready.
Train this on your generated dataset text line crops.
See notebook for full training loop (same pattern as ELA-CNN).
CRNN saved (untrained init weights — run full training loop to train).
```

```
import torch
print(torch.cuda.is_available())
print(torch.cuda.get_device_name(0) if torch.cuda.is_available() else "CPU only")
```

```
True
Tesla T4
```

```
# ── Complete retraining — both models, GPU, saves to Drive ────────────────
from google.colab import drive
```

```
import os, io, time, random, json
import numpy as np
import torch
import torch.nn as nn
import torchvision.transforms as T
from torch.utils.data import Dataset, DataLoader
from PIL import Image, ImageDraw, ImageFont
from sklearn.metrics import roc_auc_score
from pathlib import Path

# Mount Drive
drive.mount('/content/drive', force_remount=True)
SAVE_DIR = '/content/drive/MyDrive/docguard_models'
os.makedirs(SAVE_DIR, exist_ok=True)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Device: {device}")

random.seed(42)
np.random.seed(42)

# ═══════════════════════════════════════════════════
# MODEL 1 — ELA-CNN
# ═══════════════════════════════════════════════════
print("\n" + "="*50)
print("TRAINING ELA-CNN")
print("="*50)

class ELA_CNN(nn.Module):
    def __init__(self, num_classes=2):
        super().__init__()
        self.block1 = self._b(3,   32)
        self.block2 = self._b(32,  64)
        self.block3 = self._b(64,  128)
        self.block4 = self._b(128, 256)
        self.block5 = self._b(256, 512)
        self.gap = nn.AdaptiveAvgPool2d(1)
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(512,256), nn.ReLU(inplace=True), nn.Dropout(0.4),
            nn.Linear(256,num_classes)
        )
    def _b(self, i, o):
        return nn.Sequential(
            nn.Conv2d(i,o,3,padding=1,bias=False), nn.BatchNorm2d(o), nn.ReLU(inplace=True),
            nn.Conv2d(o,o,3,padding=1,bias=False), nn.BatchNorm2d(o), nn.ReLU(inplace=True),
            nn.MaxPool2d(2)
```

```
        )
    def forward(self, x):
        for b in [self.block1,self.block2,self.block3,self.block4,self.block5]: x=b(x)
        return self.classifier(self.gap(x))

def compute_ela(img, quality=90, scale=10):
    img = img.convert("RGB")
    buf = io.BytesIO()
    img.save(buf, format="JPEG", quality=quality)
    buf.seek(0)
    comp = Image.open(buf).copy()
    ela  = np.abs(np.array(img,dtype=np.float32)-np.array(comp,dtype=np.float32))*scale
    return Image.fromarray(np.clip(ela,0,255).astype(np.uint8))

# Regenerate real docs if needed
real_dir = Path('/content/real_docs')
real_dir.mkdir(exist_ok=True)
if len(list(real_dir.glob('*.jpg'))) < 500:
    print("Regenerating documents...")
    for i in range(500):
        img  = Image.new('RGB',(224,224),(252,251,248))
        draw = ImageDraw.Draw(img)
        draw.rectangle([0,0,224,40],fill=(240,240,235))
        y=50
        for _ in range(random.randint(5,10)):
            lw=random.randint(40,200)
            draw.rectangle([10,y,10+lw,y+random.randint(4,8)],fill=(random.randint(30,80),)*3)
            y+=random.randint(10,18)
            if y>200: break
        arr=np.array(img,dtype=np.float32)+np.random.normal(0,3,(224,224,3))
        Image.fromarray(np.clip(arr,0,255).astype(np.uint8)).save(real_dir/f'doc_{i:05d}.jpg',quality=92)
    print(f"✓ {i+1} docs generated")

# Build ELA dataset inline
class ELADataset(Dataset):
    def __init__(self, items, transform=None):
        self.items = items
        self.transform = transform
    def __len__(self): return len(self.items)
    def __getitem__(self, idx):
        item  = self.items[idx]
        img   = Image.open(item['file']).convert('RGB').resize((224,224))
        ela   = compute_ela(img)
        label = 0 if item['type'] == 'authentic' else 1
        if self.transform: ela = self.transform(ela)
        return ela, label
```

```
# Generate forgeries inline
print("Generating forgeries...")
out = Path('/content/dataset2')
for cls in ['authentic','text_replacement']:
    (out/'images'/cls).mkdir(parents=True, exist_ok=True)

src_paths = list(real_dir.glob('*.jpg'))
manifest  = []

for i, src in enumerate(src_paths[:400]):
    img = Image.open(src).convert('RGB').resize((224,224))
    fp  = out/'images'/'authentic'/f'a_{i:04d}.jpg'
    img.save(fp, quality=92)
    manifest.append({'file':str(fp),'type':'authentic'})

for i, src in enumerate(src_paths[:400]):
    img  = Image.open(src).convert('RGB').resize((224,224))
    draw = ImageDraw.Draw(img)
    x,y  = random.randint(10,100), random.randint(10,100)
    draw.rectangle([x,y,x+60,y+14], fill=(255,255,255))
    draw.text((x+2,y+2), random.choice(["$99,999","2019","PAID"]),
              fill=(10,10,10), font=ImageFont.load_default())
    fp = out/'images'/'text_replacement'/f'f_{i:04d}.jpg'
    img.save(fp, quality=88)
    manifest.append({'file':str(fp),'type':'text_replacement'})

random.shuffle(manifest)
n = len(manifest)
train_items = manifest[:int(n*0.8)]
val_items   = manifest[int(n*0.8):]

ela_transform = T.Compose([
    T.Resize((224,224)), T.ToTensor(),
    T.Normalize([0.485,0.456,0.406],[0.229,0.224,0.225])
])

train_loader = DataLoader(ELADataset(train_items, ela_transform), batch_size=32, shuffle=True,  num_workers=2)
val_loader   = DataLoader(ELADataset(val_items,   ela_transform), batch_size=32, shuffle=False, num_workers=2)

print(f"Train: {len(train_items)} | Val: {len(val_items)}")

ela_model = ELA_CNN().to(device)
optimizer  = torch.optim.AdamW(ela_model.parameters(), lr=1e-3, weight_decay=1e-4)
criterion  = nn.CrossEntropyLoss()
best_acc   = 0.0

for epoch in range(1, 16):  # 15 epochs — converges fast
```

```
    ela_model.train()
    for imgs, labels in train_loader:
        imgs, labels = imgs.to(device), labels.to(device)
        optimizer.zero_grad()
        criterion(ela_model(imgs), labels).backward()
        optimizer.step()

    ela_model.eval()
    correct = total = 0
    all_probs, all_labels = [], []
    with torch.no_grad():
        for imgs, labels in val_loader:
            out   = ela_model(imgs.to(device))
            probs = torch.softmax(out,1)[:,1].cpu().numpy()
            correct += (out.argmax(1).cpu().numpy() == labels.numpy()).sum()
            total   += len(labels)
            all_probs.extend(probs); all_labels.extend(labels.numpy())

    acc = correct/total*100
    auc = roc_auc_score(all_labels, all_probs)
    print(f"Epoch {epoch:02d}/15 | Acc: {acc:.1f}% | AUC: {auc:.4f}")

    if acc > best_acc:
        best_acc = acc
        torch.save({
            'model_state_dict': ela_model.state_dict(),
            'val_acc': acc, 'val_auc': auc,
            'architecture': 'ELA_CNN_v1'
        }, f'{SAVE_DIR}/ela_cnn_best.pt')
        print(f"  ✓ Saved (acc: {acc:.1f}%)")

print(f"\nELA-CNN done. Best: {best_acc:.2f}%")

# ═══════════════════════════════════════════════════
# MODEL 2 — Font CRNN
# ═══════════════════════════════════════════════════
print("\n" + "="*50)
print("TRAINING FONT CRNN")
print("="*50)

class FontCRNN(nn.Module):
    def __init__(self, num_classes=2):
        super().__init__()
        self.cnn = nn.Sequential(
            nn.Conv2d(1,32,3,padding=1), nn.BatchNorm2d(32), nn.ReLU(), nn.MaxPool2d((2,1)),
            nn.Conv2d(32,64,3,padding=1), nn.BatchNorm2d(64), nn.ReLU(), nn.MaxPool2d((2,1)),
            nn.Conv2d(64,128,3,padding=1), nn.BatchNorm2d(128), nn.ReLU(),
```

```
            nn.AdaptiveAvgPool2d((1,None)),
        )
        self.lstm = nn.LSTM(128,256,num_layers=2,bidirectional=True,batch_first=True,dropout=0.3)
        self.classifier = nn.Sequential(
            nn.Linear(512,128), nn.ReLU(), nn.Dropout(0.4), nn.Linear(128,num_classes)
        )
    def forward(self, x):
        f = self.cnn(x).squeeze(2).permute(0,2,1)
        out,_ = self.lstm(f)
        return self.classifier((out[:,0,:]+out[:,-1,:])/2)

TEXTS   = ["Invoice No: INV-2022-00423","Total: $12,500.00","Date: 15 March 2022",
           "Authorised by: J. Williams","Payment Terms: Net 30"]
INSERTS = ["125,000","2019","APPROVED","PAID","VOID"]

def auth_line():
    img=Image.new("L",(256,32),255); draw=ImageDraw.Draw(img)
    draw.text((4,8),random.choice(TEXTS)[:35],fill=20,font=ImageFont.load_default())
    arr=np.array(img,dtype=np.float32)+np.random.normal(0,2,(32,256))
    return Image.fromarray(np.clip(arr,0,255).astype(np.uint8))

def forge_line():
    img=Image.new("L",(256,32),255); draw=ImageDraw.Draw(img)
    font=ImageFont.load_default()
    t=random.choice(TEXTS); words=t.split(); sp=random.randint(1,max(1,len(words)-1))
    draw.text((4,6)," ".join(words[:sp])[:20],fill=20,font=font)
    x=4+len(" ".join(words[:sp]))*6
    draw.text((x,11),random.choice(INSERTS),fill=10,font=font)
    arr=np.array(img,dtype=np.float32)+np.random.normal(0,2,(32,256))
    return Image.fromarray(np.clip(arr,0,255).astype(np.uint8))

class FontDS(Dataset):
    def __init__(self,n,t=None): self.n=n; self.t=t
    def __len__(self): return self.n
    def __getitem__(self,i):
        label=i%2
        img=(auth_line() if label==0 else forge_line()).resize((256,32))
        if self.t: img=self.t(img)
        return img,label

ft = T.Compose([T.ToTensor(), T.Normalize([0.5],[0.5])])
tl = DataLoader(FontDS(8000,ft), batch_size=64, shuffle=True,  num_workers=2)
vl = DataLoader(FontDS(1000,ft), batch_size=64, shuffle=False, num_workers=2)

crnn      = FontCRNN().to(device)
opt2      = torch.optim.Adam(crnn.parameters(), lr=5e-4)
sch2      = torch.optim.lr_scheduler.CosineAnnealingLR(opt2, T_max=10)
```

```
best_acc2 = 0.0

for epoch in range(1, 11):  # 10 epochs — converges by epoch 2
    crnn.train()
    for imgs,labels in tl:
        imgs,labels=imgs.to(device),labels.to(device)
        opt2.zero_grad()
        criterion(crnn(imgs),labels).backward()
        torch.nn.utils.clip_grad_norm_(crnn.parameters(),1.0)
        opt2.step()
    sch2.step()

    crnn.eval()
    correct=total=0; ap,al=[],[]
    with torch.no_grad():
        for imgs,labels in vl:
            out=crnn(imgs.to(device))
            probs=torch.softmax(out,1)[:,1].cpu().numpy()
            correct+=(out.argmax(1).cpu().numpy()==labels.numpy()).sum()
            total+=len(labels); ap.extend(probs); al.extend(labels.numpy())

    acc=correct/total*100; auc=roc_auc_score(al,ap)
    print(f"Epoch {epoch:02d}/10 | Acc: {acc:.1f}% | AUC: {auc:.4f}")

    if acc>best_acc2:
        best_acc2=acc
        torch.save({
            'model_state_dict':crnn.state_dict(),
            'val_acc':acc,'val_auc':auc,'architecture':'FontCRNN_v1'
        }, f'{SAVE_DIR}/font_crnn_best.pt')
        print(f"  ✓ Saved (acc: {acc:.1f}%)")

print(f"\nFont CRNN done. Best: {best_acc2:.2f}%")

# ── Final summary ──────────────────────────────────────────────────
print("\n" + "="*50)
print("ALL MODELS COMPLETE")
print("="*50)
for fname in ['ela_cnn_best.pt','font_crnn_best.pt']:
    path=f'{SAVE_DIR}/{fname}'
    if os.path.exists(path):
        print(f"✓ {fname} — {os.path.getsize(path)/1024/1024:.1f} MB")
print(f"\nSaved to: {SAVE_DIR}")
```

```
Mounted at /content/drive
Device: cuda
```

```
==================================================
TRAINING ELA-CNN
==================================================
Regenerating documents...
✓ 500 docs generated
Generating forgeries...
Train: 640 | Val: 160
Epoch 01/15 | Acc: 51.2% | AUC: 0.3957
  ✓ Saved (acc: 51.2%)
Epoch 02/15 | Acc: 94.4% | AUC: 1.0000
  ✓ Saved (acc: 94.4%)
Epoch 03/15 | Acc: 51.2% | AUC: 0.9423
Epoch 04/15 | Acc: 100.0% | AUC: 1.0000
  ✓ Saved (acc: 100.0%)
Epoch 05/15 | Acc: 100.0% | AUC: 1.0000
Epoch 06/15 | Acc: 100.0% | AUC: 1.0000
Epoch 07/15 | Acc: 100.0% | AUC: 1.0000
Epoch 08/15 | Acc: 100.0% | AUC: 1.0000
Epoch 09/15 | Acc: 100.0% | AUC: 1.0000
Epoch 10/15 | Acc: 100.0% | AUC: 1.0000
Epoch 11/15 | Acc: 100.0% | AUC: 1.0000
Epoch 12/15 | Acc: 100.0% | AUC: 1.0000
Epoch 13/15 | Acc: 100.0% | AUC: 1.0000
Epoch 14/15 | Acc: 100.0% | AUC: 1.0000
Epoch 15/15 | Acc: 100.0% | AUC: 1.0000

ELA-CNN done. Best: 100.00%

==================================================
TRAINING FONT CRNN
==================================================
Epoch 01/10 | Acc: 100.0% | AUC: 1.0000
  ✓ Saved (acc: 100.0%)
Epoch 02/10 | Acc: 100.0% | AUC: 1.0000
Epoch 03/10 | Acc: 100.0% | AUC: 1.0000
Epoch 04/10 | Acc: 100.0% | AUC: 1.0000
Epoch 05/10 | Acc: 100.0% | AUC: 1.0000
Epoch 06/10 | Acc: 100.0% | AUC: 1.0000
Epoch 07/10 | Acc: 100.0% | AUC: 1.0000
Epoch 08/10 | Acc: 100.0% | AUC: 1.0000
Epoch 09/10 | Acc: 100.0% | AUC: 1.0000
Epoch 10/10 | Acc: 100.0% | AUC: 1.0000

Font CRNN done. Best: 100.00%

==================================================
ALL MODELS COMPLETE
==================================================
✓ ela_cnn_best.pt — 18.5 MB
✓ font_crnn_best.pt — 9.7 MB
```

```
Saved to: /content/drive/MyDrive/docguard_models
```

```
# ── Metadata MLP Training ──────────────────────────────────────────────
import torch, torch.nn as nn, numpy as np, json, os, joblib
from torch.utils.data import TensorDataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, roc_auc_score
import random

random.seed(42); np.random.seed(42)

SAVE_DIR = '/content/drive/MyDrive/docguard_models'

FEATURE_NAMES = [
    "creation_year", "modified_year", "content_date_year",
    "content_creation_delta_yrs", "modified_creation_delta_days",
    "producer_is_photoshop", "producer_is_scanner", "producer_is_acrobat",
    "producer_version_year", "software_year_doc_year_gap",
    "author_blank", "author_length", "title_blank", "xmp_docinfo_mismatch",
    "has_javascript", "font_count", "mixed_font_types", "compression_layers"
]

# ── 18 features each — verified ────────────────────────────────────────
def authentic():
    yr = random.randint(2015, 2023)
    sc = yr + random.choice([0, 0, 0, 1])
    return [
        sc,                         # creation_year
        sc,                         # modified_year
        yr,                         # content_date_year
        sc - yr,                    # content_creation_delta_yrs
        random.randint(0, 30),      # modified_creation_delta_days
        0,                          # producer_is_photoshop
        1,                          # producer_is_scanner
        random.choice([0, 1]),      # producer_is_acrobat
        sc,                         # producer_version_year
        0,                          # software_year_doc_year_gap
        random.choice([0, 0, 0, 1]), # author_blank
        random.randint(5, 40),      # author_length
        random.choice([0, 0, 1]),   # title_blank
        0,                          # xmp_docinfo_mismatch
        0,                          # has_javascript
        random.randint(1, 3),       # font_count
        0,                          # mixed_font_types
```

```
        1,                          # compression_layers
    ]

def forged_date():   # YOUR EXACT EXAMPLE — doc says 2022, PDF created 2025
    cy = random.randint(2014, 2021)
    cr = random.randint(cy + 2, 2025)
    return [
        cr,                         # creation_year
        cr,                         # modified_year
        cy,                         # content_date_year
        cr - cy,                    # content_creation_delta_yrs  ← BIG = red flag
        random.randint(0, 5),       # modified_creation_delta_days
        random.choice([0, 1]),      # producer_is_photoshop
        0,                          # producer_is_scanner
        random.choice([0, 1]),      # producer_is_acrobat
        cr,                         # producer_version_year
        cr - cy,                    # software_year_doc_year_gap  ← red flag
        random.choice([0, 1, 1]),   # author_blank
        random.randint(0, 15),      # author_length
        random.choice([0, 1]),      # title_blank
        random.choice([0, 1]),      # xmp_docinfo_mismatch
        0,                          # has_javascript
        random.randint(1, 4),       # font_count
        random.choice([0, 1]),      # mixed_font_types
        random.choice([1, 2]),      # compression_layers
    ]

def forged_photoshop():
    yr = random.randint(2018, 2024)
    return [
        yr,                         # creation_year
        yr,                         # modified_year
        yr - random.randint(0, 3),  # content_date_year
        random.randint(0, 3),       # content_creation_delta_yrs
        random.randint(0, 60),      # modified_creation_delta_days
        1,                          # producer_is_photoshop  ← red flag
        0,                          # producer_is_scanner
        0,                          # producer_is_acrobat
        yr,                         # producer_version_year
        0,                          # software_year_doc_year_gap
        1,                          # author_blank
        0,                          # author_length
        random.choice([0, 1]),      # title_blank
        random.choice([0, 1]),      # xmp_docinfo_mismatch
        0,                          # has_javascript
        random.randint(2, 6),       # font_count
        1,                          # mixed_font_types
```

```
        1,                          # compression_layers
    ]

def forged_wiped():
    yr = random.randint(2018, 2024)
    return [
        yr,                         # creation_year
        yr + random.randint(0, 2),  # modified_year
        yr - random.randint(2, 6),  # content_date_year
        random.randint(3, 8),       # content_creation_delta_yrs
        random.randint(200, 1000),  # modified_creation_delta_days
        0,                          # producer_is_photoshop
        0,                          # producer_is_scanner
        0,                          # producer_is_acrobat
        0,                          # producer_version_year  ← wiped
        random.randint(3, 8),       # software_year_doc_year_gap
        1,                          # author_blank  ← wiped
        0,                          # author_length
        1,                          # title_blank  ← wiped
        1,                          # xmp_docinfo_mismatch  ← layers disagree
        0,                          # has_javascript
        random.randint(1, 3),       # font_count
        random.choice([0, 1]),      # mixed_font_types
        2,                          # compression_layers
    ]

# ── Generate dataset ──────────────────────────────────────────────────
print("Generating 20,000 metadata records...")
records = []
for _ in range(10000):
    records.append(authentic() + [0])      # label 0 = authentic
for i in range(10000):
    fn = [forged_date, forged_photoshop, forged_wiped][i % 3]
    records.append(fn() + [1])             # label 1 = forged

random.shuffle(records)
records = np.array(records, dtype=np.float32)
X = records[:, :-1]
y = records[:, -1].astype(np.int64)

# Verify shape
assert X.shape[1] == 18, f"Expected 18 features, got {X.shape[1]}"
print(f"Dataset: {len(X)} records | Features: {X.shape[1]}")
print(f"Authentic: {sum(y==0)} | Forged: {sum(y==1)}")

# ── Train/test split + scale ──────────────────────────────────────────
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42, stratify=y)

scaler  = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

train_ds = TensorDataset(torch.tensor(X_train), torch.tensor(y_train))
test_ds  = TensorDataset(torch.tensor(X_test),  torch.tensor(y_test))
tl = DataLoader(train_ds, batch_size=256, shuffle=True)
vl = DataLoader(test_ds,  batch_size=256)

# ── Model ──────────────────────────────────────────────────────────
class MetadataMLP(nn.Module):
    def __init__(self, input_dim=18):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 128), nn.BatchNorm1d(128), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(128, 256),       nn.BatchNorm1d(256), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(256, 128),       nn.BatchNorm1d(128), nn.ReLU(), nn.Dropout(0.2),
            nn.Linear(128, 64),        nn.BatchNorm1d(64),  nn.ReLU(),
            nn.Linear(64, 2)
        )
    def forward(self, x): return self.net(x)

device    = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model     = MetadataMLP(input_dim=18).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, patience=5)

print(f"Device: {device}")
print(f"Training Metadata MLP for 50 epochs...")
print()

best_auc = 0.0
for epoch in range(1, 51):
    model.train()
    for xb, yb in tl:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        criterion(model(xb), yb).backward()
        optimizer.step()

    model.eval()
    all_preds, all_probs, all_labels = [], [], []
    with torch.no_grad():
        for xb, yb in vl:
```

```
            out = model(xb.to(device))
            all_probs.extend(torch.softmax(out, 1)[:, 1].cpu().numpy())
            all_preds.extend(out.argmax(1).cpu().numpy())
            all_labels.extend(yb.numpy())

    acc = np.mean(np.array(all_preds) == np.array(all_labels)) * 100
    auc = roc_auc_score(all_labels, all_probs)
    scheduler.step(1 - auc)

    if epoch % 10 == 0:
        print(f"Epoch {epoch:02d}/50 | Acc: {acc:.1f}% | AUC: {auc:.4f}")

    if auc > best_auc:
        best_auc = auc
        torch.save(model.state_dict(), f'{SAVE_DIR}/metadata_mlp.pt')

# ── Save scaler + feature names ──────────────────────────────────────
joblib.dump(scaler, f'{SAVE_DIR}/metadata_scaler.pkl')
with open(f'{SAVE_DIR}/metadata_features.json', 'w') as f:
    json.dump(FEATURE_NAMES, f)

# ── Final results ────────────────────────────────────────────────────
print(f"\n=== Metadata MLP Final Results ===")
print(classification_report(all_labels, all_preds,
                            target_names=['Authentic', 'Forged']))
print(f"Best AUC: {best_auc:.4f}")
print()

# ── Confirm all files saved ──────────────────────────────────────────
print("=== Files saved to Google Drive ===")
for fname in ['ela_cnn_best.pt', 'font_crnn_best.pt', 'metadata_mlp.pt',
              'metadata_scaler.pkl', 'metadata_features.json']:
    path = f'{SAVE_DIR}/{fname}'
    if os.path.exists(path):
        kb = os.path.getsize(path) / 1024
        print(f"  ✓ {fname} — {kb:.0f} KB")
    else:
        print(f"  ✗ {fname} — NOT FOUND")

print("\nALL THREE MODELS COMPLETE!")
```

```
Generating 20,000 metadata records...
Dataset: 20000 records | Features: 18
Authentic: 10000 | Forged: 10000
Device: cuda
Training Metadata MLP for 50 epochs...
```

```
Epoch 10/50 | Acc: 100.0% | AUC: 1.0000
Epoch 20/50 | Acc: 100.0% | AUC: 1.0000
Epoch 30/50 | Acc: 100.0% | AUC: 1.0000
Epoch 40/50 | Acc: 100.0% | AUC: 1.0000
Epoch 50/50 | Acc: 100.0% | AUC: 1.0000

=== Metadata MLP Final Results ===
              precision    recall  f1-score   support

   Authentic       1.00      1.00      1.00      2000
      Forged       1.00      1.00      1.00      2000

    accuracy                           1.00      4000
   macro avg       1.00      1.00      1.00      4000
weighted avg       1.00      1.00      1.00      4000

Best AUC: 1.0000

=== Files saved to Google Drive ===
  ✓ ela_cnn_best.pt — 18967 KB
  ✓ font_crnn_best.pt — 9882 KB
  ✓ metadata_mlp.pt — 319 KB
  ✓ metadata_scaler.pkl — 1 KB
  ✓ metadata_features.json — 0 KB

ALL THREE MODELS COMPLETE!
```

```
# ── Cell A: Load all models ──────────────────────────────────────────────
import os, io, json, base64, hashlib, re, time
import numpy as np
import torch
import torch.nn as nn
import torchvision.transforms as T
from PIL import Image
import cv2
import joblib
from google.colab import drive

drive.mount('/content/drive', force_remount=True)

SAVE_DIR = '/content/drive/MyDrive/docguard_models'
device   = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Device: {device}")

# ── Model definitions ────────────────────────────────────────────────────
class ELA_CNN(nn.Module):
    def __init__(self, num_classes=2):
        super().__init__()
```

```
        self.block1 = self._b(3,   32)
        self.block2 = self._b(32,  64)
        self.block3 = self._b(64,  128)
        self.block4 = self._b(128, 256)
        self.block5 = self._b(256, 512)
        self.gap = nn.AdaptiveAvgPool2d(1)
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(512,256), nn.ReLU(inplace=True), nn.Dropout(0.4),
            nn.Linear(256,num_classes)
        )
    def _b(self, i, o):
        return nn.Sequential(
            nn.Conv2d(i,o,3,padding=1,bias=False), nn.BatchNorm2d(o), nn.ReLU(inplace=True),
            nn.Conv2d(o,o,3,padding=1,bias=False), nn.BatchNorm2d(o), nn.ReLU(inplace=True),
            nn.MaxPool2d(2)
        )
    def forward(self, x):
        for b in [self.block1,self.block2,self.block3,self.block4,self.block5]: x=b(x)
        return self.classifier(self.gap(x))

class MetadataMLP(nn.Module):
    def __init__(self, input_dim=18):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim,128), nn.BatchNorm1d(128), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(128,256),       nn.BatchNorm1d(256), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(256,128),       nn.BatchNorm1d(128), nn.ReLU(), nn.Dropout(0.2),
            nn.Linear(128,64),        nn.BatchNorm1d(64),  nn.ReLU(),
            nn.Linear(64,2)
        )
    def forward(self, x): return self.net(x)

class FontCRNN(nn.Module):
    def __init__(self, num_classes=2):
        super().__init__()
        self.cnn = nn.Sequential(
            nn.Conv2d(1,32,3,padding=1), nn.BatchNorm2d(32), nn.ReLU(), nn.MaxPool2d((2,1)),
            nn.Conv2d(32,64,3,padding=1), nn.BatchNorm2d(64), nn.ReLU(), nn.MaxPool2d((2,1)),
            nn.Conv2d(64,128,3,padding=1), nn.BatchNorm2d(128), nn.ReLU(),
            nn.AdaptiveAvgPool2d((1,None)),
        )
        self.lstm = nn.LSTM(128,256,num_layers=2,bidirectional=True,batch_first=True,dropout=0.3)
        self.classifier = nn.Sequential(
            nn.Linear(512,128), nn.ReLU(), nn.Dropout(0.4), nn.Linear(128,num_classes)
        )
    def forward(self, x):
```

```
        f = self.cnn(x).squeeze(2).permute(0,2,1)
        out,_ = self.lstm(f)
        return self.classifier((out[:,0,:]+out[:,-1,:])/2)

# ── Load weights ──────────────────────────────────────────────────────
ela_model = ELA_CNN().to(device)
ela_model.load_state_dict(
    torch.load(f'{SAVE_DIR}/ela_cnn_best.pt',
               map_location=device, weights_only=False)['model_state_dict'])
ela_model.eval()
print("✓ ELA-CNN loaded")

meta_model = MetadataMLP(input_dim=18).to(device)
meta_model.load_state_dict(
    torch.load(f'{SAVE_DIR}/metadata_mlp.pt',
               map_location=device, weights_only=False))
meta_model.eval()
meta_scaler = joblib.load(f'{SAVE_DIR}/metadata_scaler.pkl')
with open(f'{SAVE_DIR}/metadata_features.json') as f:
    feature_names = json.load(f)
print("✓ Metadata MLP loaded")

crnn_model = FontCRNN().to(device)
crnn_model.load_state_dict(
    torch.load(f'{SAVE_DIR}/font_crnn_best.pt',
               map_location=device, weights_only=False)['model_state_dict'])
crnn_model.eval()
print("✓ Font CRNN loaded")

print(f"\n✓ All models loaded on {device}!")
```

```
Mounted at /content/drive
Device: cuda
✓ ELA-CNN loaded
✓ Metadata MLP loaded
✓ Font CRNN loaded

✓ All models loaded on cuda!
```

```
!pip install -q PyMuPDF reportlab
import fitz
print("Ready:", fitz.__version__)
```

```
Ready: 1.27.2.2
```

```
# ── Cell B: Analysis engine ──────────────────────────────────────────
val_transform = T.Compose([
    T.Resize((224,224)), T.ToTensor(),
    T.Normalize([0.485,0.456,0.406],[0.229,0.224,0.225])
])

def compute_ela(img, quality=90, scale=10):
    img = img.convert("RGB")
    buf = io.BytesIO()
    img.save(buf, format="JPEG", quality=quality)
    buf.seek(0)
    comp = Image.open(buf).copy()
    ela  = np.abs(np.array(img,dtype=np.float32)-np.array(comp,dtype=np.float32))*scale
    return Image.fromarray(np.clip(ela,0,255).astype(np.uint8))

def run_ela(img):
    ela    = compute_ela(img)
    tensor = val_transform(ela).unsqueeze(0).to(device)
    activations, gradients = [], []
    fh = ela_model.block5.register_forward_hook(
        lambda m,i,o: activations.append(o.detach()))
    bh = ela_model.block5.register_full_backward_hook(
        lambda m,gi,go: gradients.append(go[0].detach()))
    tensor_g = tensor.clone().requires_grad_(True)
    out = ela_model(tensor_g)
    ela_model.zero_grad()
    out[0,1].backward()
    fh.remove(); bh.remove()

    with torch.no_grad():
        probs = torch.softmax(ela_model(tensor), dim=1)
        score = int(probs[0,1].item() * 100)

    heatmap_b64 = None
    try:
        weights = gradients[0].mean(dim=[2,3],keepdim=True)
        cam     = torch.relu((weights*activations[0]).sum(dim=1)).squeeze().cpu().numpy()
        if cam.max() > cam.min():
            cam = (cam-cam.min())/(cam.max()-cam.min())
        orig_np = np.array(img.convert("RGB").resize((224,224)))
        cam_r   = cv2.resize(cam,(224,224))
        heatmap = cv2.applyColorMap((cam_r*255).astype(np.uint8), cv2.COLORMAP_JET)
        overlay = (orig_np*0.6 + cv2.cvtColor(heatmap,cv2.COLOR_BGR2RGB)*0.4).astype(np.uint8)
        buf2    = io.BytesIO()
        Image.fromarray(overlay).save(buf2, format="PNG")
        heatmap_b64 = "" + base64.b64encode(buf2.getvalue()).decode()
    except:
```

```
            pass

        return {"score": score, "heatmap": heatmap_b64,
                "signal": "ELA compression artifact analysis"}

    def run_metadata(file_bytes):
        signals  = []
        risk     = 0
        features = {}
        try:
            import fitz
            doc  = fitz.open(stream=file_bytes, filetype="pdf")
            meta = doc.metadata
            text = "".join(p.get_text() for p in doc.pages())[:3000]

            creation_year = modified_year = content_year = 0
            cd = meta.get("creationDate","") or ""
            md = meta.get("modDate","")      or ""
            if len(cd)>=8:
                try: creation_year = int(cd[2:6])
                except: pass
            if len(md)>=8:
                try: modified_year = int(md[2:6])
                except: pass

            years = re.findall(r'\b(19|20)\d{2}\b', text)
            if years:
                content_year = min(int(y) for y in years if 1990<=int(y)<=2030)

            producer = (meta.get("producer","") or "").lower()
            author   = (meta.get("author","")   or "").strip()

            features = {
                "creation_year": creation_year,
                "modified_year": modified_year,
                "content_year":  content_year,
                "author":        author[:60] if author else "(blank)",
                "producer":      meta.get("producer","unknown")[:80],
            }

            if content_year > 0 and creation_year > 0:
                delta = creation_year - content_year
                if delta > 1:
                    risk += 35
                    signals.append({"severity":"HIGH",
                        "signal":"Date mismatch — backdating detected",
                        "detail":f"Content references {content_year}, PDF created {creation_year} ({delta}yr gap)"})
```

```
        if "photoshop" in producer:
            risk += 30
            signals.append({"severity":"HIGH",
                "signal":"Photoshop detected on document",
                "detail":f"Producer: {meta.get('producer','')[:60]}"})

        if len(author) == 0:
            risk += 10
            signals.append({"severity":"MEDIUM",
                "signal":"Author field blank",
                "detail":"Author metadata missing — may be deliberately wiped"})

        if doc.get_js():
            risk += 25
            signals.append({"severity":"HIGH",
                "signal":"JavaScript embedded in PDF",
                "detail":"PDF contains JavaScript — can modify displayed content"})

        if not signals:
            signals.append({"severity":"OK",
                "signal":"Metadata appears consistent",
                "detail":f"Creation: {creation_year}, Content: {content_year}"})

        doc.close()

        feat_vec = np.array([[
            creation_year, modified_year, content_year,
            abs(creation_year-content_year), 0,
            1 if "photoshop" in producer else 0,
            0, 1, creation_year,
            abs(creation_year-content_year),
            1 if len(author)==0 else 0, len(author),
            0, 0, 0, 1, 0, 1
        ]], dtype=np.float32)
        scaled = meta_scaler.transform(feat_vec)
        tensor = torch.tensor(scaled).to(device)
        with torch.no_grad():
            probs    = torch.softmax(meta_model(tensor), dim=1)
            ml_score = int(probs[0,1].item() * 100)
        risk = int(risk*0.55 + ml_score*0.45)

    except Exception as e:
        signals.append({"severity":"INFO",
            "signal":"Image file — metadata limited",
            "detail":str(e)[:80]})
```

```
    return {"score": min(100,risk), "signals": signals, "features": features}

def run_noise(img):
    arr  = np.array(img.convert("L"), dtype=np.float32)
    h,w  = arr.shape
    bh,bw = max(1,h//8), max(1,w//8)
    variances = [np.var(arr[r:r+bh,c:c+bw])
                 for r in range(0,h-bh,bh)
                 for c in range(0,w-bw,bw)]
    variances = np.array(variances) if variances else np.array([0.0])
    cv    = variances.std()/(variances.mean()+1e-7)
    score = min(100, int(cv*55)) if cv>0.4 else 0
    return {"score": score, "noise_cv": round(float(cv),4),
            "signal": "High noise heterogeneity" if score>40
                      else "Noise pattern consistent"}

def analyze(file_bytes, filename="document"):
    t0     = time.time()
    is_pdf = file_bytes[:4] == b"%PDF"
    try:
        if is_pdf:
            import fitz
            pdf = fitz.open(stream=file_bytes, filetype="pdf")
            pix = pdf[0].get_pixmap(matrix=fitz.Matrix(2,2))
            img = Image.frombytes("RGB",[pix.width,pix.height],pix.samples)
            pdf.close()
        else:
            img = Image.open(io.BytesIO(file_bytes)).convert("RGB")
    except Exception as e:
        return {"error": str(e)}

    ela   = run_ela(img)
    meta  = run_metadata(file_bytes)
    noise = run_noise(img)

    risk = int(meta["score"]*0.42 + ela["score"]*0.35 + noise["score"]*0.23)
    risk = min(100, risk)

    if   risk >= 70: verdict,color,icon = "HIGH RISK — Likely Forged",        "red",    "🔴"
    elif risk >= 40: verdict,color,icon = "MEDIUM RISK — Suspicious",          "orange", "🟠"
    elif risk >= 20: verdict,color,icon = "LOW RISK — Minor Anomalies",        "yellow", "🟡"
    else:            verdict,color,icon = "AUTHENTIC — No Tampering Detected",  "green",  "🟢"

    return {
        "filename":           filename,
        "file_hash":          hashlib.md5(file_bytes).hexdigest()[:12],
        "analysis_time":      round(time.time()-t0, 2),
```

```
        "timestamp":          time.strftime("%Y-%m-%dT%H:%M:%SZ"),
        "forgery_risk_score": risk,
        "authenticity_score": 100-risk,
        "verdict":            verdict,
        "verdict_color":      color,
        "verdict_icon":       icon,
        "stream_scores": {
            "metadata_risk": meta["score"],
            "pixel_risk":    ela["score"],
            "noise_risk":    noise["score"],
        },
        "streams": {
            "pixel_analysis":    {"score": ela["score"],
                                  "signal": ela["signal"],
                                  "heatmap": ela["heatmap"]},
            "metadata_analysis": {"score": meta["score"],
                                  "signals": meta["signals"],
                                  "features": meta["features"]},
            "noise_analysis":    {"score": noise["score"],
                                  "signal": noise["signal"],
                                  "noise_cv": noise["noise_cv"]},
        }
    }

print("✓ Analysis engine ready")
```

```
✓ Analysis engine ready
```

```
import subprocess, time
subprocess.run(['fuser', '-k', '8000/tcp'], capture_output=True)
time.sleep(3)
print("✓ Server stopped")
```

```
# ── Cell C: FastAPI + ngrok server ──────────────────────────────────────
!pip install -q fastapi uvicorn pyngrok nest-asyncio python-multipart

import nest_asyncio
import uvicorn
import threading
from fastapi import FastAPI, File, UploadFile
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse
from pyngrok import ngrok, conf

nest_asyncio.apply()
```

```
# ── ngrok token ──────────────────────────────────────────────────────
conf.get_default().auth_token = "3BcAdGewY6yDLLPL1uE6jV7SnxV_2gduPD3nn2BAbHFw2arJy"

# ── FastAPI app ──────────────────────────────────────────────────────
app = FastAPI(title="DocGuard AI")
app.add_middleware(CORSMiddleware,
                   allow_origins=["*"],
                   allow_methods=["*"],
                   allow_headers=["*"])

@app.get("/")
def root():
    return {"service": "DocGuard AI", "status": "running"}

@app.get("/health")
def health():
    return {"status": "ok", "models": "all loaded", "device": str(device)}

@app.post("/analyze")
async def analyze_doc(file: UploadFile = File(...)):
    file_bytes = await file.read()
    if len(file_bytes) > 50*1024*1024:
        return JSONResponse({"error": "File too large. Max 50MB."}, status_code=400)
    result = analyze(file_bytes, file.filename or "document")
    return JSONResponse(result)

# ── Open ngrok tunnel ────────────────────────────────────────────────
tunnel     = ngrok.connect(8000, "http")
PUBLIC_URL = tunnel.public_url

print(f"")
print(f"{'='*55}")
print(f"  DocGuard AI is LIVE!")
print(f"  Public URL: {PUBLIC_URL}")
print(f"{'='*55}")
print(f"")
print(f"  Next step:")
print(f"  Open frontend/index.html in your browser")
print(f"  Replace API_URL with: {PUBLIC_URL}")
print(f"")
print(f"  Or test directly:")
print(f"  {PUBLIC_URL}/health")

# ── Start server (keeps running) ─────────────────────────────────────
def run_server():
    uvicorn.run(app, host="0.0.0.0", port=8000, log_level="warning")
```

```
server_thread = threading.Thread(target=run_server, daemon=True)
server_thread.start()

import time
time.sleep(2)
print(f"\nServer is running. Do NOT close this cell.")
print(f"Keep this tab open while using the web app.")
```

```
========================================================
  DocGuard AI is LIVE!
  Public URL: https://albert-multinuclear-evita.ngrok-free.dev
========================================================

  Next step:
  Open frontend/index.html in your browser
  Replace API_URL with: https://albert-multinuclear-evita.ngrok-free.dev

  Or test directly:
  https://albert-multinuclear-evita.ngrok-free.dev/health

Server is running. Do NOT close this cell.
Keep this tab open while using the web app.
```

```
import requests, io, json
from reportlab.pdfgen import canvas
from reportlab.lib.pagesizes import A4

buf = io.BytesIO()
c   = canvas.Canvas(buf, pagesize=A4)
c.setFont("Helvetica", 12)
c.drawString(100, 750, "INVOICE")
c.drawString(100, 720, "Date: 15 March 2019")
c.drawString(100, 695, "Amount: $12,500.00")
c.save()
buf.seek(0)

r    = requests.post('http://localhost:8000/analyze',
                     files={'file': ('test.pdf', buf, 'application/pdf')})
data = r.json()
print("Verdict:", data['verdict'])
print("Risk:", data['forgery_risk_score'])
for s in data['streams']['metadata_analysis']['signals']:
    print(f"  [{s['severity']}]", s['signal'])
    print("  →", s['detail'])
```

```
ERROR:    Exception in ASGI application
Traceback (most recent call last):
  File "/usr/local/lib/python3.12/dist-packages/uvicorn/protocols/http/httptools_impl.py", line 416, in run_asgi
    result = await app(  # type: ignore[func-returns-value]
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/uvicorn/middleware/proxy_headers.py", line 60, in __call__
    return await self.app(scope, receive, send)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.12/dist-packages/fastapi/applications.py", line 1160, in __call__
    await super().__call__(scope, receive, send)
  File "/usr/local/lib/python3.12/dist-packages/starlette/applications.py", line 107, in __call__
    await self.middleware_stack(scope, receive, send)
  File "/usr/local/lib/python3.12/dist-packages/starlette/middleware/errors.py", line 186, in __call__
    raise exc
  File "/usr/local/lib/python3.12/dist-packages/starlette/middleware/errors.py", line 164, in __call__
    await self.app(scope, receive, _send)
  File "/usr/local/lib/python3.12/dist-packages/starlette/middleware/cors.py", line 87, in __call__
    await self.app(scope, receive, send)
  File "/usr/local/lib/python3.12/dist-packages/starlette/middleware/exceptions.py", line 63, in __call__
    await wrap_app_handling_exceptions(self.app, conn)(scope, receive, send)
  File "/usr/local/lib/python3.12/dist-packages/starlette/_exception_handler.py", line 53, in wrapped_app
    raise exc
  File "/usr/local/lib/python3.12/dist-packages/starlette/_exception_handler.py", line 42, in wrapped_app
    await app(scope, receive, sender)
  File "/usr/local/lib/python3.12/dist-packages/fastapi/middleware/asyncexitstack.py", line 18, in __call__
    await self.app(scope, receive, send)
```

```
# ── All-in-one: analysis engine + server ──────────────────────────────
import os, io, json, base64, hashlib, re, time
import numpy as np
import torch
import torch.nn as nn
import torchvision.transforms as T
from PIL import Image
import cv2, joblib
import nest_asyncio, uvicorn, threading
from fastapi import FastAPI, File, UploadFile
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse
from pyngrok import ngrok, conf

nest_asyncio.apply()

SAVE_DIR = '/content/drive/MyDrive/docguard_models'
device   = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# ── Model definitions ─────────────────────────────────────────────────
class ELA_CNN(nn.Module):
    def __init__(self, num_classes=2):
```

```
        super().__init__()
        self.block1 = self._b(3,   32)
        self.block2 = self._b(32,  64)
        self.block3 = self._b(64,  128)
        self.block4 = self._b(128, 256)
        self.block5 = self._b(256, 512)
        self.gap = nn.AdaptiveAvgPool2d(1)
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(512,256), nn.ReLU(inplace=True), nn.Dropout(0.4),
            nn.Linear(256,num_classes))
    def _b(self, i, o):
        return nn.Sequential(
            nn.Conv2d(i,o,3,padding=1,bias=False), nn.BatchNorm2d(o), nn.ReLU(inplace=True),
            nn.Conv2d(o,o,3,padding=1,bias=False), nn.BatchNorm2d(o), nn.ReLU(inplace=True),
            nn.MaxPool2d(2))
    def forward(self, x):
        for b in [self.block1,self.block2,self.block3,self.block4,self.block5]: x=b(x)
        return self.classifier(self.gap(x))

class MetadataMLP(nn.Module):
    def __init__(self, input_dim=18):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim,128), nn.BatchNorm1d(128), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(128,256),       nn.BatchNorm1d(256), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(256,128),       nn.BatchNorm1d(128), nn.ReLU(), nn.Dropout(0.2),
            nn.Linear(128,64),        nn.BatchNorm1d(64),  nn.ReLU(),
            nn.Linear(64,2))
    def forward(self, x): return self.net(x)

class FontCRNN(nn.Module):
    def __init__(self, num_classes=2):
        super().__init__()
        self.cnn = nn.Sequential(
            nn.Conv2d(1,32,3,padding=1), nn.BatchNorm2d(32), nn.ReLU(), nn.MaxPool2d((2,1)),
            nn.Conv2d(32,64,3,padding=1), nn.BatchNorm2d(64), nn.ReLU(), nn.MaxPool2d((2,1)),
            nn.Conv2d(64,128,3,padding=1), nn.BatchNorm2d(128), nn.ReLU(),
            nn.AdaptiveAvgPool2d((1,None)))
        self.lstm = nn.LSTM(128,256,num_layers=2,bidirectional=True,batch_first=True,dropout=0.3)
        self.classifier = nn.Sequential(
            nn.Linear(512,128), nn.ReLU(), nn.Dropout(0.4), nn.Linear(128,num_classes))
    def forward(self, x):
        f = self.cnn(x).squeeze(2).permute(0,2,1)
        out,_ = self.lstm(f)
        return self.classifier((out[:,0,:]+out[:,-1,:])/2)
```

```
# ── Load models ──────────────────────────────────────────────────────
print("Loading models...")
ela_model = ELA_CNN().to(device)
ela_model.load_state_dict(torch.load(f'{SAVE_DIR}/ela_cnn_best.pt',
    map_location=device, weights_only=False)['model_state_dict'])
ela_model.eval()

meta_model = MetadataMLP(18).to(device)
meta_model.load_state_dict(torch.load(f'{SAVE_DIR}/metadata_mlp.pt',
    map_location=device, weights_only=False))
meta_model.eval()
meta_scaler = joblib.load(f'{SAVE_DIR}/metadata_scaler.pkl')

crnn_model = FontCRNN().to(device)
crnn_model.load_state_dict(torch.load(f'{SAVE_DIR}/font_crnn_best.pt',
    map_location=device, weights_only=False)['model_state_dict'])
crnn_model.eval()
print(f"✓ All models loaded on {device}")

# ── Analysis functions ───────────────────────────────────────────────
val_transform = T.Compose([T.Resize((224,224)), T.ToTensor(),
    T.Normalize([0.485,0.456,0.406],[0.229,0.224,0.225])])

def compute_ela(img, quality=90, scale=10):
    img=img.convert("RGB"); buf=io.BytesIO()
    img.save(buf,format="JPEG",quality=quality); buf.seek(0)
    comp=Image.open(buf).copy()
    ela=np.abs(np.array(img,dtype=np.float32)-np.array(comp,dtype=np.float32))*scale
    return Image.fromarray(np.clip(ela,0,255).astype(np.uint8))

def run_ela(img):
    tensor=val_transform(compute_ela(img)).unsqueeze(0).to(device)
    activations, gradients = [], []
    fh=ela_model.block5.register_forward_hook(lambda m,i,o: activations.append(o.detach()))
    bh=ela_model.block5.register_full_backward_hook(lambda m,gi,go: gradients.append(go[0].detach()))
    tensor_g=tensor.clone().requires_grad_(True)
    out=ela_model(tensor_g); ela_model.zero_grad(); out[0,1].backward()
    fh.remove(); bh.remove()
    with torch.no_grad():
        score=int(torch.softmax(ela_model(tensor),dim=1)[0,1].item()*100)
    heatmap_b64=None
    try:
        weights=gradients[0].mean(dim=[2,3],keepdim=True)
        cam=torch.relu((weights*activations[0]).sum(dim=1)).squeeze().cpu().numpy()
        if cam.max()>cam.min(): cam=(cam-cam.min())/(cam.max()-cam.min())
        orig_np=np.array(img.convert("RGB").resize((224,224)))
        cam_r=cv2.resize(cam,(224,224))
```

```
        heatmap=cv2.applyColorMap((cam_r*255).astype(np.uint8),cv2.COLORMAP_JET)
        overlay=(orig_np*0.6+cv2.cvtColor(heatmap,cv2.COLOR_BGR2RGB)*0.4).astype(np.uint8)
        buf2=io.BytesIO(); Image.fromarray(overlay).save(buf2,format="PNG")
        heatmap_b64=""+base64.b64encode(buf2.getvalue()).decode()
    except: pass
    return {"score":score,"heatmap":heatmap_b64,"signal":"ELA compression artifact analysis"}

def run_metadata(file_bytes):
    signals=[]; risk=0; features={}
    try:
        import fitz
        doc=fitz.open(stream=file_bytes,filetype="pdf")
        meta=doc.metadata
        text="".join(p.get_text() for p in doc.pages())[:3000]
        creation_year=modified_year=content_year=0
        cd=meta.get("creationDate","") or ""
        md=meta.get("modDate","")      or ""
        if len(cd)>=8:
            try: creation_year=int(cd[2:6])
            except: pass
        if len(md)>=8:
            try: modified_year=int(md[2:6])
            except: pass
        # Find years in content — broad pattern catches INV-2019-001 etc
        years=[int(m) for m in re.findall(r'(?<!\d)(19|20)\d{2}(?!\d)',text)]
        years=[y for y in years if 1990<=y<=2030]
        content_year=min(years) if years else 0
        producer=(meta.get("producer","") or "").lower()
        author  =(meta.get("author","")   or "").strip()
        features={"creation_year":creation_year,"modified_year":modified_year,
                  "content_year":content_year,
                  "author":author[:60] if author else "(blank)",
                  "producer":meta.get("producer","unknown")[:80]}
        # Signal checks
        if content_year>0 and creation_year>0:
            delta=creation_year-content_year
            if delta>1:
                risk+=35
                signals.append({"severity":"HIGH",
                    "signal":"Date mismatch — backdating detected",
                    "detail":f"Content references {content_year}, PDF created {creation_year} ({delta}yr gap)"})
            elif delta>0:
                risk+=12
                signals.append({"severity":"MEDIUM",
                    "signal":"Date gap detected",
                    "detail":f"Content year {content_year} predates creation {creation_year}"})
        if "photoshop" in producer:
```

```
            risk+=30
            signals.append({"severity":"HIGH","signal":"Photoshop detected",
                "detail":f"Producer: {meta.get('producer','')[:60]}"})
        if len(author)==0:
            risk+=10
            signals.append({"severity":"MEDIUM","signal":"Author field blank",
                "detail":"Author metadata missing — may be deliberately wiped"})
        if doc.get_js():
            risk+=25
            signals.append({"severity":"HIGH","signal":"JavaScript in PDF",
                "detail":"PDF contains JavaScript — can modify content"})
        if not signals:
            signals.append({"severity":"OK","signal":"Metadata consistent",
                "detail":f"Creation: {creation_year}, Content: {content_year}"})
        doc.close()
        # MLP refinement
        feat_vec=np.array([[creation_year,modified_year,content_year,
            abs(creation_year-content_year),0,
            1 if "photoshop" in producer else 0,
            0,1,creation_year,abs(creation_year-content_year),
            1 if len(author)==0 else 0,len(author),
            0,0,0,1,0,1]],dtype=np.float32)
        scaled=meta_scaler.transform(feat_vec)
        with torch.no_grad():
            ml_score=int(torch.softmax(meta_model(torch.tensor(scaled).to(device)),dim=1)[0,1].item()*100)
        risk=int(risk*0.55+ml_score*0.45)
    except Exception as e:
        signals.append({"severity":"INFO","signal":"Image file — metadata limited",
            "detail":str(e)[:80]})
    return {"score":min(100,risk),"signals":signals,"features":features}

def run_noise(img):
    arr=np.array(img.convert("L"),dtype=np.float32); h,w=arr.shape
    bh,bw=max(1,h//8),max(1,w//8)
    v=[np.var(arr[r:r+bh,c:c+bw]) for r in range(0,h-bh,bh) for c in range(0,w-bw,bw)]
    v=np.array(v) if v else np.array([0.0])
    cv=v.std()/(v.mean()+1e-7)
    return {"score":min(100,int(cv*55)) if cv>0.4 else 0,
            "noise_cv":round(float(cv),4),
            "signal":"High noise heterogeneity" if cv>0.4 else "Noise consistent"}

def analyze(file_bytes, filename="document"):
    t0=time.time()
    is_pdf=file_bytes[:4]==b"%PDF"
    try:
        if is_pdf:
            import fitz
```

```
            pdf=fitz.open(stream=file_bytes,filetype="pdf")
            pix=pdf[0].get_pixmap(matrix=fitz.Matrix(2,2))
            img=Image.frombytes("RGB",[pix.width,pix.height],pix.samples)
            pdf.close()
        else:
            img=Image.open(io.BytesIO(file_bytes)).convert("RGB")
    except Exception as e:
        return {"error":str(e)}
    ela=run_ela(img); meta=run_metadata(file_bytes); noise=run_noise(img)
    risk=min(100,int(meta["score"]*0.42+ela["score"]*0.35+noise["score"]*0.23))
    if   risk>=70: v,c,i="HIGH RISK — Likely Forged","red","🔴"
    elif risk>=40: v,c,i="MEDIUM RISK — Suspicious","orange","🟠"
    elif risk>=20: v,c,i="LOW RISK — Minor Anomalies","yellow","🟡"
    else:          v,c,i="AUTHENTIC — No Tampering Detected","green","🟢"
    return {"filename":filename,"file_hash":hashlib.md5(file_bytes).hexdigest()[:12],
            "analysis_time":round(time.time()-t0,2),
            "timestamp":time.strftime("%Y-%m-%dT%H:%M:%SZ"),
            "forgery_risk_score":risk,"authenticity_score":100-risk,
            "verdict":v,"verdict_color":c,"verdict_icon":i,
            "stream_scores":{"metadata_risk":meta["score"],"pixel_risk":ela["score"],"noise_risk":noise["score"]},
            "streams":{"pixel_analysis":{"score":ela["score"],"signal":ela["signal"],"heatmap":ela["heatmap"]},
                       "metadata_analysis":{"score":meta["score"],"signals":meta["signals"],"features":meta["features"]},
                       "noise_analysis":{"score":noise["score"],"signal":noise["signal"],"noise_cv":noise["noise_cv"]}}}

# ── Kill old server ──────────────────────────────────────────────
import subprocess
subprocess.run(['fuser','-k','8000/tcp'], capture_output=True)
time.sleep(2)

# ── Start FastAPI ────────────────────────────────────────────────
conf.get_default().auth_token = "3BcAdGewY6yDLLPL1uE6jV7SnxV_2gduPD3nn2BAbHFw2arJy"

app = FastAPI(title="DocGuard AI")
app.add_middleware(CORSMiddleware,allow_origins=["*"],allow_methods=["*"],allow_headers=["*"])

@app.get("/health")
def health(): return {"status":"ok","models":"all loaded","device":str(device)}

@app.post("/analyze")
async def analyze_doc(file: UploadFile = File(...)):
    return JSONResponse(analyze(await file.read(), file.filename or "document"))

tunnel     = ngrok.connect(8000,"http")
PUBLIC_URL = tunnel.public_url

print(f"\n{'='*50}")
print(f"  DocGuard AI LIVE: {PUBLIC_URL}")
```

```
print(f"{'='*50}")
print(f"  Health: {PUBLIC_URL}/health")

def run_server():
    uvicorn.run(app,host="0.0.0.0",port=8000,log_level="warning")

threading.Thread(target=run_server,daemon=True).start()
time.sleep(2)
print("  Server running. Keep this cell active.")
```

```
Loading models...
✓ All models loaded on cuda

==================================================
  DocGuard AI LIVE: https://albert-multinuclear-evita.ngrok-free.dev
==================================================
  Health: https://albert-multinuclear-evita.ngrok-free.dev/health
  Server running. Keep this cell active.
```

```
import requests, io, json
from reportlab.pdfgen import canvas
from reportlab.lib.pagesizes import A4

buf = io.BytesIO()
c   = canvas.Canvas(buf, pagesize=A4)
c.setFont("Helvetica", 12)
c.drawString(100, 750, "INVOICE")
c.drawString(100, 720, "Date: 15 March 2019")
c.drawString(100, 695, "Invoice No: INV-2019-00423")
c.drawString(100, 670, "Amount Due: $12,500.00")
c.drawString(100, 645, "Authorised by: J. Williams")
c.save()
buf.seek(0)

r    = requests.post('http://localhost:8000/analyze',
                     files={'file': ('invoice_2019.pdf', buf, 'application/pdf')})
data = r.json()

print(f"Verdict:  {data['verdict_icon']} {data['verdict']}")
print(f"Risk:     {data['forgery_risk_score']}/100")
print(f"\nStream scores:")
print(f"  Metadata: {data['stream_scores']['metadata_risk']}/100")
print(f"  Pixel:    {data['stream_scores']['pixel_risk']}/100")
print(f"  Noise:    {data['stream_scores']['noise_risk']}/100")
print(f"\nSignals:")
for s in data['streams']['metadata_analysis']['signals']:
```

```
    print(f"  [{s['severity']}] {s['signal']}")
    print(f"           {s['detail']}")
```

```
Verdict:  🟡 MEDIUM RISK — Suspicious
Risk:     58/100

Stream scores:
  Metadata: 0/100
  Pixel:    100/100
  Noise:    100/100

Signals:
  [INFO] Image file — metadata limited
           'Document' object has no attribute 'get_js'
```

```
# ═══════════════════════════════════════════════════════════════
# DOCGUARD AI — COMPLETE FINAL VERSION
# Run this ONE cell. Everything works.
# ═══════════════════════════════════════════════════════════════
import subprocess, time
subprocess.run(['fuser','-k','8000/tcp'], capture_output=True)
time.sleep(2)

!pip install -q PyMuPDF reportlab

import os, io, json, base64, hashlib, re, time
import numpy as np
import torch
import torch.nn as nn
import torchvision.transforms as T
from PIL import Image
import cv2, joblib
import nest_asyncio, uvicorn, threading
from fastapi import FastAPI, File, UploadFile
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse
from pyngrok import ngrok, conf
import fitz

nest_asyncio.apply()

SAVE_DIR = '/content/drive/MyDrive/docguard_models'
device   = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# ── Models ──────────────────────────────────────────────────────
class ELA_CNN(nn.Module):
    def __init__(self):
```

```
        super().__init__()
        self.block1=self._b(3,32);self.block2=self._b(32,64)
        self.block3=self._b(64,128);self.block4=self._b(128,256);self.block5=self._b(256,512)
        self.gap=nn.AdaptiveAvgPool2d(1)
        self.classifier=nn.Sequential(nn.Flatten(),nn.Linear(512,256),nn.ReLU(inplace=True),nn.Dropout(0.4),nn.Linear(256,2))
    def _b(self,i,o):
        return nn.Sequential(
            nn.Conv2d(i,o,3,padding=1,bias=False),nn.BatchNorm2d(o),nn.ReLU(inplace=True),
            nn.Conv2d(o,o,3,padding=1,bias=False),nn.BatchNorm2d(o),nn.ReLU(inplace=True),nn.MaxPool2d(2))
    def forward(self,x):
        for b in [self.block1,self.block2,self.block3,self.block4,self.block5]:x=b(x)
        return self.classifier(self.gap(x))

class MetadataMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net=nn.Sequential(
            nn.Linear(18,128),nn.BatchNorm1d(128),nn.ReLU(),nn.Dropout(0.3),
            nn.Linear(128,256),nn.BatchNorm1d(256),nn.ReLU(),nn.Dropout(0.3),
            nn.Linear(256,128),nn.BatchNorm1d(128),nn.ReLU(),nn.Dropout(0.2),
            nn.Linear(128,64),nn.BatchNorm1d(64),nn.ReLU(),nn.Linear(64,2))
    def forward(self,x):return self.net(x)

class FontCRNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.cnn=nn.Sequential(
            nn.Conv2d(1,32,3,padding=1),nn.BatchNorm2d(32),nn.ReLU(),nn.MaxPool2d((2,1)),
            nn.Conv2d(32,64,3,padding=1),nn.BatchNorm2d(64),nn.ReLU(),nn.MaxPool2d((2,1)),
            nn.Conv2d(64,128,3,padding=1),nn.BatchNorm2d(128),nn.ReLU(),nn.AdaptiveAvgPool2d((1,None)))
        self.lstm=nn.LSTM(128,256,num_layers=2,bidirectional=True,batch_first=True,dropout=0.3)
        self.classifier=nn.Sequential(nn.Linear(512,128),nn.ReLU(),nn.Dropout(0.4),nn.Linear(128,2))
    def forward(self,x):
        f=self.cnn(x).squeeze(2).permute(0,2,1);out,_=self.lstm(f)
        return self.classifier((out[:,0,:]+out[:,-1,:])/2)

# ── Load weights ──────────────────────────────────────────────────────
print("Loading models...")
ela=ELA_CNN().to(device)
ela.load_state_dict(torch.load(f'{SAVE_DIR}/ela_cnn_best.pt',map_location=device,weights_only=False)['model_state_dict'])
ela.eval()

mlp=MetadataMLP().to(device)
mlp.load_state_dict(torch.load(f'{SAVE_DIR}/metadata_mlp.pt',map_location=device,weights_only=False))
mlp.eval()
scaler=joblib.load(f'{SAVE_DIR}/metadata_scaler.pkl')
```

```
crnn=FontCRNN().to(device)
crnn.load_state_dict(torch.load(f'{SAVE_DIR}/font_crnn_best.pt',map_location=device,weights_only=False)['model_state_dict'])
crnn.eval()
print(f"✓ Models loaded on {device}")

# ── Transform ──────────────────────────────────────────────────────────
tfm=T.Compose([T.Resize((224,224)),T.ToTensor(),T.Normalize([0.485,0.456,0.406],[0.229,0.224,0.225])])

def ela_img(img,q=90,s=10):
    img=img.convert("RGB");b=io.BytesIO();img.save(b,format="JPEG",quality=q);b.seek(0)
    c=Image.open(b).copy()
    e=np.abs(np.array(img,dtype=np.float32)-np.array(c,dtype=np.float32))*s
    return Image.fromarray(np.clip(e,0,255).astype(np.uint8))

# ── ELA stream ─────────────────────────────────────────────────────────
def stream_ela(img):
    t=tfm(ela_img(img)).unsqueeze(0).to(device)
    acts,grads=[],[]
    fh=ela.block5.register_forward_hook(lambda m,i,o:acts.append(o.detach()))
    bh=ela.block5.register_full_backward_hook(lambda m,gi,go:grads.append(go[0].detach()))
    tg=t.clone().requires_grad_(True);out=ela(tg);ela.zero_grad();out[0,1].backward()
    fh.remove();bh.remove()
    with torch.no_grad():score=int(torch.softmax(ela(t),1)[0,1].item()*100)
    hm=None
    try:
        w=grads[0].mean(dim=[2,3],keepdim=True)
        cam=torch.relu((w*acts[0]).sum(1)).squeeze().cpu().numpy()
        if cam.max()>cam.min():cam=(cam-cam.min())/(cam.max()-cam.min())
        orig=np.array(img.convert("RGB").resize((224,224)))
        cr=cv2.resize(cam,(224,224))
        h=cv2.applyColorMap((cr*255).astype(np.uint8),cv2.COLORMAP_JET)
        ov=(orig*0.6+cv2.cvtColor(h,cv2.COLOR_BGR2RGB)*0.4).astype(np.uint8)
        b2=io.BytesIO();Image.fromarray(ov).save(b2,format="PNG")
        hm=""+base64.b64encode(b2.getvalue()).decode()
    except:pass
    return {"score":score,"heatmap":hm,"signal":"ELA compression analysis"}

# ── Metadata stream ────────────────────────────────────────────────────
def stream_meta(fb):
    sigs=[];risk=0;feat={}
    try:
        doc=fitz.open(stream=fb,filetype="pdf")
        meta=doc.metadata
        text="".join(p.get_text() for p in doc.pages())[:3000]
        cy=my=ky=0
        cd=meta.get("creationDate","") or ""
        md=meta.get("modDate","")      or ""
```

```
if len(cd)>=8:
    try:cy=int(cd[2:6])
    except:pass
if len(md)>=8:
    try:my=int(md[2:6])
    except:pass
yrs=[int(x) for x in re.findall(r'(?<!\d)((?:19|20)\d{2})(?!\d)',text) if 1990<=int(x)<=2030]
ky=min(yrs) if yrs else 0
prod=(meta.get("producer","") or "").lower()
auth=(meta.get("author","")   or "").strip()
feat={"creation_year":cy,"modified_year":my,"content_year":ky,
      "author":auth[:60] if auth else "(blank)",
      "producer":meta.get("producer","unknown")[:80]}
# Date mismatch — key signal
if ky>0 and cy>0:
    d=cy-ky
    if d>1:
        risk+=35
        sigs.append({"severity":"HIGH",
            "signal":"Date mismatch — backdating detected",
            "detail":f"Content references {ky}, PDF created {cy} ({d}yr gap)"})
    elif d>0:
        risk+=12
        sigs.append({"severity":"MEDIUM","signal":"Date gap",
            "detail":f"Content year {ky} predates creation {cy}"})
if "photoshop" in prod:
    risk+=30
    sigs.append({"severity":"HIGH","signal":"Photoshop detected",
        "detail":f"Producer: {meta.get('producer','')[:60]}"})
if len(auth)==0:
    risk+=10
    sigs.append({"severity":"MEDIUM","signal":"Author field blank",
        "detail":"Author metadata missing"})
# JavaScript check — compatible with all fitz versions
has_js=False
try:
    for i in range(len(doc)):
        if "/JavaScript" in doc[i].get_text("rawdict").get("blocks",""):
            has_js=True;break
except:pass
try:
    if hasattr(doc,'get_javascript') and doc.get_javascript():has_js=True
except:pass
try:
    if hasattr(doc,'get_js') and doc.get_js():has_js=True
except:pass
if has_js:
```

```
            risk+=25
            sigs.append({"severity":"HIGH","signal":"JavaScript in PDF","detail":"PDF contains JavaScript"})
        if not sigs:
            sigs.append({"severity":"OK","signal":"Metadata consistent",
                "detail":f"Creation: {cy}, Content: {ky}"})
        doc.close()
        fv=np.array([[cy,my,ky,abs(cy-ky),0,1 if "photoshop" in prod else 0,
            0,1,cy,abs(cy-ky),1 if len(auth)==0 else 0,len(auth),0,0,0,1,0,1]],dtype=np.float32)
        sc=scaler.transform(fv)
        with torch.no_grad():
            ms=int(torch.softmax(mlp(torch.tensor(sc).to(device)),1)[0,1].item()*100)
        risk=int(risk*0.55+ms*0.45)
    except Exception as e:
        sigs.append({"severity":"INFO","signal":"Image file — metadata limited","detail":str(e)[:80]})
    return {"score":min(100,risk),"signals":sigs,"features":feat}

# ── Noise stream ──────────────────────────────────────────────────────
def stream_noise(img):
    a=np.array(img.convert("L"),dtype=np.float32);h,w=a.shape
    bh,bw=max(1,h//8),max(1,w//8)
    v=[np.var(a[r:r+bh,c:c+bw]) for r in range(0,h-bh,bh) for c in range(0,w-bw,bw)]
    v=np.array(v) if v else np.array([0.0])
    cv=v.std()/(v.mean()+1e-7)
    return {"score":min(100,int(cv*55)) if cv>0.4 else 0,
            "noise_cv":round(float(cv),4),
            "signal":"High noise heterogeneity" if cv>0.4 else "Noise consistent"}

# ── Main analyze ──────────────────────────────────────────────────────
def analyze(fb,fn="document"):
    t0=time.time()
    try:
        if fb[:4]==b"%PDF":
            pdf=fitz.open(stream=fb,filetype="pdf")
            pix=pdf[0].get_pixmap(matrix=fitz.Matrix(2,2))
            img=Image.frombytes("RGB",[pix.width,pix.height],pix.samples)
            pdf.close()
        else:
            img=Image.open(io.BytesIO(fb)).convert("RGB")
    except Exception as e:
        return {"error":str(e)}
    e=stream_ela(img);m=stream_meta(fb);n=stream_noise(img)
    risk=min(100,int(m["score"]*0.42+e["score"]*0.35+n["score"]*0.23))
    if   risk>=70:v,c,i="HIGH RISK — Likely Forged","red","🔴"
    elif risk>=40:v,c,i="MEDIUM RISK — Suspicious","orange","🟠"
    elif risk>=20:v,c,i="LOW RISK — Minor Anomalies","yellow","🟡"
    else:         v,c,i="AUTHENTIC — No Tampering Detected","green","🟢"
    return {"filename":fn,"file_hash":hashlib.md5(fb).hexdigest()[:12],
```

```
            "analysis_time":round(time.time()-t0,2),
            "timestamp":time.strftime("%Y-%m-%dT%H:%M:%SZ"),
            "forgery_risk_score":risk,"authenticity_score":100-risk,
            "verdict":v,"verdict_color":c,"verdict_icon":i,
            "stream_scores":{"metadata_risk":m["score"],"pixel_risk":e["score"],"noise_risk":n["score"]},
            "streams":{"pixel_analysis":{"score":e["score"],"signal":e["signal"],"heatmap":e["heatmap"]},
                       "metadata_analysis":{"score":m["score"],"signals":m["signals"],"features":m["features"]},
                       "noise_analysis":{"score":n["score"],"signal":n["signal"],"noise_cv":n["noise_cv"]}}}

# ── FastAPI ──────────────────────────────────────────────────────────
conf.get_default().auth_token="3BcAdGewY6yDLLPL1uE6jV7SnxV_2gduPD3nn2BAbHFw2arJy"
app=FastAPI(title="DocGuard AI")
app.add_middleware(CORSMiddleware,allow_origins=["*"],allow_methods=["*"],allow_headers=["*"])

@app.get("/health")
def health():return{"status":"ok","models":"all loaded","device":str(device)}

@app.post("/analyze")
async def analyze_doc(file:UploadFile=File(...)):
    return JSONResponse(analyze(await file.read(),file.filename or "document"))

tunnel=ngrok.connect(8000,"http")
URL=tunnel.public_url
print(f"\n{'='*50}")
print(f"  DocGuard AI LIVE: {URL}")
print(f"{'='*50}")
print(f"  Health check: {URL}/health")
print(f"  Keep this cell running.")

threading.Thread(target=lambda:uvicorn.run(app,host="0.0.0.0",port=8000,log_level="warning"),daemon=True).start()
time.sleep(3)

# ── Self-test ────────────────────────────────────────────────────────
print("\nRunning self-test with backdated PDF...")
from reportlab.pdfgen import canvas as rlcanvas
from reportlab.lib.pagesizes import A4
buf=io.BytesIO()
cv2_=rlcanvas.Canvas(buf,pagesize=A4)
cv2_.setFont("Helvetica",12)
cv2_.drawString(100,750,"INVOICE")
cv2_.drawString(100,720,"Date: 15 March 2019")
cv2_.drawString(100,695,"Invoice No: INV-2019-00423")
cv2_.drawString(100,670,"Amount: $12,500.00")
cv2_.save();buf.seek(0)
r=analyze(buf.read(),"invoice_2019.pdf")
print(f"Verdict:  {r['verdict_icon']} {r['verdict']}")
print(f"Risk:     {r['forgery_risk_score']}/100")
```

```
print(f"Metadata: {r['stream_scores']['metadata_risk']}/100")
for s in r['streams']['metadata_analysis']['signals']:
    print(f"  [{s['severity']}] {s['signal']} — {s['detail']}")
print("\n✓ System fully operational")
```

```
Loading models...
✓ Models loaded on cuda

==================================================
  DocGuard AI LIVE: https://albert-multinuclear-evita.ngrok-free.dev
==================================================
  Health check: https://albert-multinuclear-evita.ngrok-free.dev/health
  Keep this cell running.

Running self-test with backdated PDF...
Verdict:  🔴 HIGH RISK — Likely Forged
Risk:     84/100
Metadata: 63/100
  [HIGH] Date mismatch — backdating detected — Content references 2019, PDF created 2026 (7yr gap)

✓ System fully operational
```

```
# ── Push final working code to HuggingFace Space ──────────────────────
from huggingface_hub import HfApi

HF_TOKEN   = "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"
SPACE_ID   = "pavanusd/docguard-ai-app"
api        = HfApi()

final_app = '''
import os, io, json, base64, hashlib, re, time
import numpy as np
import torch
import torch.nn as nn
import torchvision.transforms as T
from PIL import Image
import cv2, joblib
from fastapi import FastAPI, File, UploadFile
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse
from huggingface_hub import hf_hub_download
import fitz

HF_REPO = "pavanusd/docguard-ai"

class ELA_CNN(nn.Module):
    def __init__(self):
        super().__init__()
```

```
        self.block1=self._b(3,32);self.block2=self._b(32,64)
        self.block3=self._b(64,128);self.block4=self._b(128,256);self.block5=self._b(256,512)
        self.gap=nn.AdaptiveAvgPool2d(1)
        self.classifier=nn.Sequential(nn.Flatten(),nn.Linear(512,256),nn.ReLU(inplace=True),nn.Dropout(0.4),nn.Linear(256,2))
    def _b(self,i,o):
        return nn.Sequential(
            nn.Conv2d(i,o,3,padding=1,bias=False),nn.BatchNorm2d(o),nn.ReLU(inplace=True),
            nn.Conv2d(o,o,3,padding=1,bias=False),nn.BatchNorm2d(o),nn.ReLU(inplace=True),nn.MaxPool2d(2))
    def forward(self,x):
        for b in [self.block1,self.block2,self.block3,self.block4,self.block5]:x=b(x)
        return self.classifier(self.gap(x))

class MetadataMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net=nn.Sequential(
            nn.Linear(18,128),nn.BatchNorm1d(128),nn.ReLU(),nn.Dropout(0.3),
            nn.Linear(128,256),nn.BatchNorm1d(256),nn.ReLU(),nn.Dropout(0.3),
            nn.Linear(256,128),nn.BatchNorm1d(128),nn.ReLU(),nn.Dropout(0.2),
            nn.Linear(128,64),nn.BatchNorm1d(64),nn.ReLU(),nn.Linear(64,2))
    def forward(self,x):return self.net(x)

class FontCRNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.cnn=nn.Sequential(
            nn.Conv2d(1,32,3,padding=1),nn.BatchNorm2d(32),nn.ReLU(),nn.MaxPool2d((2,1)),
            nn.Conv2d(32,64,3,padding=1),nn.BatchNorm2d(64),nn.ReLU(),nn.MaxPool2d((2,1)),
            nn.Conv2d(64,128,3,padding=1),nn.BatchNorm2d(128),nn.ReLU(),nn.AdaptiveAvgPool2d((1,None)))
        self.lstm=nn.LSTM(128,256,num_layers=2,bidirectional=True,batch_first=True,dropout=0.3)
        self.classifier=nn.Sequential(nn.Linear(512,128),nn.ReLU(),nn.Dropout(0.4),nn.Linear(128,2))
    def forward(self,x):
        f=self.cnn(x).squeeze(2).permute(0,2,1);out,_=self.lstm(f)
        return self.classifier((out[:,0,:]+out[:,-1,:])/2)

print("Loading models from HuggingFace Hub...")
device=torch.device("cpu")

ela=ELA_CNN().to(device)
ela.load_state_dict(torch.load(hf_hub_download(HF_REPO,"ela_cnn_best.pt"),map_location=device,weights_only=False)["model_state_dict"])
ela.eval()

mlp=MetadataMLP().to(device)
mlp.load_state_dict(torch.load(hf_hub_download(HF_REPO,"metadata_mlp.pt"),map_location=device,weights_only=False))
mlp.eval()
scaler=joblib.load(hf_hub_download(HF_REPO,"metadata_scaler.pkl"))
```

```
crnn=FontCRNN().to(device)
crnn.load_state_dict(torch.load(hf_hub_download(HF_REPO,"font_crnn_best.pt"),map_location=device,weights_only=False)["model_state_dict"])
crnn.eval()
print("All models loaded!")

tfm=T.Compose([T.Resize((224,224)),T.ToTensor(),T.Normalize([0.485,0.456,0.406],[0.229,0.224,0.225])])

def ela_img(img,q=90,s=10):
    img=img.convert("RGB");b=io.BytesIO();img.save(b,format="JPEG",quality=q);b.seek(0)
    c=Image.open(b).copy()
    e=np.abs(np.array(img,dtype=np.float32)-np.array(c,dtype=np.float32))*s
    return Image.fromarray(np.clip(e,0,255).astype(np.uint8))

def stream_ela(img):
    t=tfm(ela_img(img)).unsqueeze(0).to(device)
    with torch.no_grad():score=int(torch.softmax(ela(t),1)[0,1].item()*100)
    return {"score":score,"heatmap":None,"signal":"ELA compression analysis"}

def stream_meta(fb):
    sigs=[];risk=0;feat={}
    try:
        doc=fitz.open(stream=fb,filetype="pdf")
        meta=doc.metadata
        text="".join(p.get_text() for p in doc.pages())[:3000]
        cy=my=ky=0
        cd=meta.get("creationDate","") or ""
        md=meta.get("modDate","")      or ""
        if len(cd)>=8:
            try:cy=int(cd[2:6])
            except:pass
        if len(md)>=8:
            try:my=int(md[2:6])
            except:pass
        yrs=[int(x) for x in re.findall(r"(?<!\\d)((?:19|20)\\d{2})(?!\\d)",text) if 1990<=int(x)<=2030]
        ky=min(yrs) if yrs else 0
        prod=(meta.get("producer","") or "").lower()
        auth=(meta.get("author","")   or "").strip()
        feat={"creation_year":cy,"modified_year":my,"content_year":ky,
              "author":auth[:60] if auth else "(blank)",
              "producer":meta.get("producer","unknown")[:80]}
        if ky>0 and cy>0:
            d=cy-ky
            if d>1:
                risk+=35
                sigs.append({"severity":"HIGH","signal":"Date mismatch — backdating detected",
                    "detail":f"Content references {ky}, PDF created {cy} ({d}yr gap)"})
            elif d>0:
```

```
                risk+=12
                sigs.append({"severity":"MEDIUM","signal":"Date gap",
                    "detail":f"Content year {ky} predates creation {cy}"})
        if "photoshop" in prod:
            risk+=30
            sigs.append({"severity":"HIGH","signal":"Photoshop detected",
                "detail":f"Producer: {meta.get('producer','')[:60]}"})
        if len(auth)==0:
            risk+=10
            sigs.append({"severity":"MEDIUM","signal":"Author field blank",
                "detail":"Author metadata missing"})
        has_js=False
        try:
            if hasattr(doc,"get_javascript") and doc.get_javascript():has_js=True
        except:pass
        try:
            if hasattr(doc,"get_js") and doc.get_js():has_js=True
        except:pass
        if has_js:
            risk+=25
            sigs.append({"severity":"HIGH","signal":"JavaScript in PDF","detail":"PDF contains JavaScript"})
        if not sigs:
            sigs.append({"severity":"OK","signal":"Metadata consistent",
                "detail":f"Creation: {cy}, Content: {ky}"})
        doc.close()
        fv=np.array([[cy,my,ky,abs(cy-ky),0,1 if "photoshop" in prod else 0,
            0,1,cy,abs(cy-ky),1 if len(auth)==0 else 0,len(auth),0,0,0,1,0,1]],dtype=np.float32)
        sc=scaler.transform(fv)
        with torch.no_grad():
            ms=int(torch.softmax(mlp(torch.tensor(sc)),1)[0,1].item()*100)
        risk=int(risk*0.55+ms*0.45)
    except Exception as e:
        sigs.append({"severity":"INFO","signal":"Image file — metadata limited","detail":str(e)[:80]})
    return {"score":min(100,risk),"signals":sigs,"features":feat}

def stream_noise(img):
    a=np.array(img.convert("L"),dtype=np.float32);h,w=a.shape
    bh,bw=max(1,h//8),max(1,w//8)
    v=[np.var(a[r:r+bh,c:c+bw]) for r in range(0,h-bh,bh) for c in range(0,w-bw,bw)]
    v=np.array(v) if v else np.array([0.0])
    cv=v.std()/(v.mean()+1e-7)
    return {"score":min(100,int(cv*55)) if cv>0.4 else 0,
            "noise_cv":round(float(cv),4),
            "signal":"High noise heterogeneity" if cv>0.4 else "Noise consistent"}

def analyze(fb,fn="document"):
    t0=time.time()
```

```
    try:
        if fb[:4]==b"%PDF":
            pdf=fitz.open(stream=fb,filetype="pdf")
            pix=pdf[0].get_pixmap(matrix=fitz.Matrix(2,2))
            img=Image.frombytes("RGB",[pix.width,pix.height],pix.samples)
            pdf.close()
        else:
            img=Image.open(io.BytesIO(fb)).convert("RGB")
    except Exception as e:
        return {"error":str(e)}
    e=stream_ela(img);m=stream_meta(fb);n=stream_noise(img)
    risk=min(100,int(m["score"]*0.42+e["score"]*0.35+n["score"]*0.23))
    if   risk>=70:v,c,i="HIGH RISK — Likely Forged","red","🔴"
    elif risk>=40:v,c,i="MEDIUM RISK — Suspicious","orange","🟠"
    elif risk>=20:v,c,i="LOW RISK — Minor Anomalies","yellow","🟡"
    else:         v,c,i="AUTHENTIC — No Tampering Detected","green","🟢"
    return {"filename":fn,"file_hash":hashlib.md5(fb).hexdigest()[:12],
            "analysis_time":round(time.time()-t0,2),
            "timestamp":time.strftime("%Y-%m-%dT%H:%M:%SZ"),
            "forgery_risk_score":risk,"authenticity_score":100-risk,
            "verdict":v,"verdict_color":c,"verdict_icon":i,
            "stream_scores":{"metadata_risk":m["score"],"pixel_risk":e["score"],"noise_risk":n["score"]},
            "streams":{"pixel_analysis":{"score":e["score"],"signal":e["signal"],"heatmap":e["heatmap"]},
                       "metadata_analysis":{"score":m["score"],"signals":m["signals"],"features":m["features"]},
                       "noise_analysis":{"score":n["score"],"signal":n["signal"],"noise_cv":n["noise_cv"]}}}

app=FastAPI(title="DocGuard AI")
app.add_middleware(CORSMiddleware,allow_origins=["*"],allow_methods=["*"],allow_headers=["*"])

@app.get("/")
def root():return{"service":"DocGuard AI","status":"running"}

@app.get("/health")
def health():return{"status":"ok","models":"all loaded","device":"cpu"}

@app.post("/analyze")
async def analyze_doc(file:UploadFile=File(...)):
    return JSONResponse(analyze(await file.read(),file.filename or "document"))
'''

with open('/tmp/app.py','w') as f:
    f.write(final_app)

api.upload_file(
    path_or_fileobj='/tmp/app.py',
    path_in_repo='app.py',
    repo_id=SPACE_ID,
```

```
    token=HF_TOKEN,
    repo_type="space",
)
print("✓ Final app.py uploaded to HuggingFace Space")
print("Space will rebuild in 2-3 minutes")
print(f"URL: https://pavanusd-docguard-ai-app.hf.space")
```

```
✓ Final app.py uploaded to HuggingFace Space
Space will rebuild in 2-3 minutes
URL: https://pavanusd-docguard-ai-app.hf.space
```

```
from huggingface_hub import HfApi

HF_TOKEN  = "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"
SPACE_ID  = "pavanusd/docguard-ai-app"
api       = HfApi()

dockerfile = """FROM python:3.11-slim
WORKDIR /app
RUN apt-get update && apt-get install -y libgl1-mesa-dev libglib2.0-0 libsm6 libxext6 libxrender-dev && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 7860
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "7860"]
"""

with open('/tmp/Dockerfile','w') as f:
    f.write(dockerfile)

api.upload_file(
    path_or_fileobj='/tmp/Dockerfile',
    path_in_repo='Dockerfile',
    repo_id=SPACE_ID,
    token=HF_TOKEN,
    repo_type="space",
)
print("✓ Dockerfile fixed and uploaded")
print("Space rebuilding now...")
print("Watch: https://huggingface.co/spaces/pavanusd/docguard-ai-app/logs")
```

```
✓ Dockerfile fixed and uploaded
Space rebuilding now...
Watch: https://huggingface.co/spaces/pavanusd/docguard-ai-app/logs
```

```
from huggingface_hub import HfApi

HF_TOKEN  = "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"
SPACE_ID  = "pavanusd/docguard-ai-app"
api       = HfApi()

# Fix requirements — pin numpy<2
requirements = """fastapi
uvicorn
"numpy<2"
torch==2.1.0
torchvision==0.16.0
Pillow
opencv-python-headless
PyMuPDF
scikit-learn==1.6.1
joblib
huggingface_hub
python-multipart
"""

with open('/tmp/requirements.txt','w') as f:
    f.write(requirements)

api.upload_file(
    path_or_fileobj='/tmp/requirements.txt',
    path_in_repo='requirements.txt',
    repo_id=SPACE_ID,
    token=HF_TOKEN,
    repo_type="space",
)
print("✓ requirements.txt fixed — Space rebuilding now")
print("Watch: https://huggingface.co/spaces/pavanusd/docguard-ai-app/logs")
```

```
✓ requirements.txt fixed — Space rebuilding now
Watch: https://huggingface.co/spaces/pavanusd/docguard-ai-app/logs
```

```
from huggingface_hub import HfApi

HF_TOKEN  = "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"
SPACE_ID  = "pavanusd/docguard-ai-app"
api       = HfApi()

# Fix requirements — pin numpy<2
requirements = """fastapi
uvicorn
```

```
"numpy<2"
torch==2.1.0
torchvision==0.16.0
Pillow
opencv-python-headless
PyMuPDF
scikit-learn==1.6.1
joblib
huggingface_hub
python-multipart
"""

with open('/tmp/requirements.txt','w') as f:
    f.write(requirements)

api.upload_file(
    path_or_fileobj='/tmp/requirements.txt',
    path_in_repo='requirements.txt',
    repo_id=SPACE_ID,
    token=HF_TOKEN,
    repo_type="space",
)
print("✓ requirements.txt fixed — Space rebuilding now")
print("Watch: https://huggingface.co/spaces/pavanusd/docguard-ai-app/logs")
```

```
No files have been modified since last commit. Skipping to prevent empty commit.
WARNING:huggingface_hub.hf_api:No files have been modified since last commit. Skipping to prevent empty commit.
✓ requirements.txt fixed — Space rebuilding now
Watch: https://huggingface.co/spaces/pavanusd/docguard-ai-app/logs
```

```
from huggingface_hub import HfApi

HF_TOKEN  = "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"
SPACE_ID  = "pavanusd/docguard-ai-app"
api       = HfApi()

# Force rebuild by updating requirements with pinned numpy
requirements = """fastapi
uvicorn
numpy==1.26.4
torch==2.1.0
torchvision==0.16.0
Pillow
opencv-python-headless
PyMuPDF
scikit-learn==1.6.1
joblib
```

```
huggingface_hub
python-multipart
"""

with open('/tmp/requirements.txt', 'w') as f:
    f.write(requirements)

api.upload_file(
    path_or_fileobj='/tmp/requirements.txt',
    path_in_repo='requirements.txt',
    repo_id=SPACE_ID,
    token=HF_TOKEN,
    repo_type="space",
    commit_message="pin numpy==1.26.4 to fix RuntimeError: Numpy is not available"
)
print("✓ Uploaded — Space is rebuilding now")
print("Watch: https://huggingface.co/spaces/pavanusd/docguard-ai-app/logs")
```

```
✓ Uploaded — Space is rebuilding now
Watch: https://huggingface.co/spaces/pavanusd/docguard-ai-app/logs
```